Code: A Chapter-by-Chapter Intensive Reading Guide

Table of Contents

Chapter 1: Best Friends

Two kids, one window, one flashlight. This is the starting point of computer communication.

There are no server rooms, no fiber optics, no satellites. There is only a pair of neighbor kids who want to chat after their parents turn off the lights, using the flashes of a flashlight to transmit letters. Between their windows, using the most primitive light, they completed the simplest "digital communication" in human history.

1.1 A Flash Is Information: The Birth of the Bit

A flashlight only has two states—on, or off. These two states are the smallest unit of information in the computer world: **1 bit**.

A bit doesn't care what you are transmitting—a love letter, a military secret, a distress signal. It only cares about one thing: "yes" or "no." On is 1, off is 0. Everything you see when you turn on a computer—text, images, music, programs—at the most fundamental level, is the result of billions of these "yes" or "no" states stacked together.

But having a flash is not enough. If the two kids didn't agree in advance that "one short flash = A, two short flashes = B," then the flash is just a flash, not information. **The essence of encoding is giving physical states semantic meaning.** The sender and receiver share a set of rules—

use the rules to turn information into signals, and then use the rules again to turn signals back into information.

If you extract the story of these two kids, you get a universal communication model:

```
[Source] → [Encoder] → [Channel] → [Decoder] → [Destination]
    ↑          ↑           ↑           ↑            ↑
 Kid A's   A's finger   Light goes    B's eyes     Kid B's
  brain    + agreed      through     + agreed      brain
            rules       the window     rules
```

Compare this model to today's network cards, fiber optics, and Wi-Fi: it's simply replacing "finger pressing a flashlight" with "network card sending an electrical signal," and "light going through a window" with "signal traveling through a network cable or optical fiber." The physical medium has changed, but the logic has not.

1.2 Three Pieces of Foreshadowing: Problems This Chapter Leaves Unsolved

This chapter leaves behind more problems than it solves. This is intentional—these problems will be unraveled one by one in the chapters to come.

1. **How do you know when a single flash is over?** The receiver stares at the flashlight and sees a flash. But how long is this "one moment"? How do you distinguish "one long flash" from "two consecutive short flashes"? This is a synchronization problem. The clock signals, start bits, and stop bits in later chapters are all designed to solve it.
2. **What if you miss a flash?** If Kid B misses a flash, and Kid A has already sent the rest, the letter B spells out will be wrong. This is an error-handling problem. The parity bits, parity checks, and CRC error-correcting codes in later chapters are designed to solve it.
3. **Does sending 'Z' really require pressing the flashlight 26 times?** "One short flash = A, twenty-six short flashes = Z"—to send the letter Z, Kid A's finger would have to press 26 times. This is an efficiency problem. Chapter 2 will solve it: using multiple flashlights simultaneously, you only need one flash per letter.

1.3 A Single Seed Grows the Whole Book

This chapter is the starting point for the entire book *Code*. Each subsequent chapter is a growth from this seed. Multiple flashlights transmitting simultaneously, information volume increases exponentially (Chapter 2). Relays replace human hands, turning the flash into an electrically controlled switch (Chapter 6). Countless relays pieced together form the CPU (Chapter 17). The operating system manages it all (Chapter 22).

Every chapter is a leaf, but the root is here. Two kids, one window, one flashlight.

Chapter 2: Codes and Combinations

One flashlight isn't enough. The two kids from Chapter 1 have already proven this—sending a 'Z' requires pressing the flashlight 26 times, and having a complete sentence would wear their fingers to the bone. But they have nothing else at hand, only the flashlight.

What if they had more than one?

2.1 Not Just Brighter, But More Possibilities

A single flashlight has only two states: on or off. Two flashlights are different. Each can be independently on or off—it's not "brighter," but "more combinations." 2 × 2 = 4 combinations: off-off, off-on, on-off, on-on.

This means raising the flashlights once can represent four different letters. It's no longer "press once to say one letter," but "raise once to say one letter."

Every time you add a flashlight, the information capacity doubles. Three flashlights have 8 combinations, four have 16, five have 32, six have 64, seven have 128—just enough to cover all English letters, numbers, and punctuation marks. Eight flashlights have 256 combinations, which is the origin of 1 byte. Every character you write is the result of 8 flashlights being simultaneously raised or lowered inside the computer.

Mathematically, this pattern is written as 2^n. 'n' is the number of flashlights, and the result is the number of symbols you can represent. Information volume grows exponentially with the number of bits—every additional bit doubles the world.

2.2 From Flashlights to Code Space

How many flashlights you possess determines the size of your "code space." The code space is the set of all your possible combinations—however many flashlights you have, that is how many slots you can assign.

7 bits, 128 slots. You can assign one slot each to 26 letters, 10 digits, several dozen punctuation marks, and a bunch of control characters. This is the origin of ASCII encoding. 32 bits, 4.2 billion slots. You could assign a unique slot to every person on Earth—this is why IPv4 addresses are insufficient and the origin of IPv6.

When you write code declaring a 32-bit signed integer, you are saying: "I want to use 32 flashlights. 1 to represent the sign, 31 to represent the value. My code space ranges from -2,147,483,648 to 2,147,483,647." This number isn't arbitrary; it is the mathematical result of all possible combinations of 32 flashlights.

2.3 Serial vs. Parallel: Two Ways to Flash

The communication method in Chapter 1 was serial—one flash after another, sent in sequence. The receiver sees a sequence of on-off states and translates them one by one. This method saves on flashlights but is slow.

The method in this chapter is parallel—multiple flashlights raised simultaneously, transmitting multiple bits at once. One time unit is enough to transmit an entire letter. Fast, but it requires more flashlights and more physical wires.

Modern computers use parallel transmission internally. Between your CPU and memory, there are 64 data lines—equivalent to 64 flashlights flashing simultaneously, transmitting 64 bits per clock cycle. But in long-distance communication, serial transmission actually has the advantage—you cannot stuff 64 optical fibers into a submarine cable to simulate 64 flashlights.

Parallel is faster, but every additional flashlight adds a physical line. Inside a chip, the wires are short, so 64 flashlights are no big deal. Inside a submarine cable, every single optical fiber comes at a huge cost. There's an even subtler problem: multiple flashlights must be raised simultaneously—if one is a fraction of a second late, the receiver will see an incorrect combination. This timing deviation is called **clock skew** in high-speed digital circuits, one of the most troublesome problems inside a modern CPU. The clock signal exists precisely to solve it.

2.4 Multiple Flashlights, or Multiple Brightness Levels?

Wait—why must we increase the number of flashlights? Could we give a flashlight more than just two states, "on" and "off," and instead have multiple brightness levels? For example: off = 0, dim = 1, medium = 2, brightest = 3—this way, one flashlight would have 4 states, and two flashlights would have 4 × 4 = 16 combinations, equivalent to the capacity of 4 two-state flashlights.

This is another direction. This book takes the route of "increasing the number of flashlights"—using many two-state flashlights, rather than a few multi-state ones. The reason is simple: two states are the easiest to distinguish. On and off, one or the other, no ambiguity. Multiple brightness levels are very hard to control precisely in the physical world—what brightness counts as "dim" when the battery is fully charged today, and what counts as "dim" tomorrow when the battery is nearly dead? The receiver would have great difficulty judging. Signals also attenuate during transmission—after traveling ten meters, a "medium" brightness might look like "dim."

Computers made the same choice: all signals are binary—either high voltage or low voltage. Not because binary is the most elegant, but because two states are the most reliable to implement and recognize. Digital circuits use a clear **threshold voltage** to define 0 and 1—below a certain voltage is 0, above a certain voltage is 1, with a "forbidden zone" in between serving as a noise margin. This simple design allows computers to maintain extremely high reliability across billions of operations.

2.5 A Seed, a Leaf

This chapter is the first leaf sprouting from the seed of Chapter 1. From one flashlight to many, from 2 states to 2^n states, from serial to parallel, from "press once" to "raise once." The following chapters will continue along this path—the combinations of these flashlights can not only represent letters but also numbers, and then perform arithmetic (Chapters 8-9); how to remember the states of these flashlights instead of

forgetting them once seen (Chapters 14-16); using these flashlights to simultaneously represent instructions and data, forming the von Neumann architecture (Chapter 17).

But the root remains those two kids, separated by a window, holding more than one flashlight.

Chapter 3: Braille and Binary Codes

Light is good, but it is not the only way. If one of the two kids were blind, the flashlight would be useless. Light switching from existence to nothingness does not exist for that person.

Information transmission cannot rely on a single sense. In 19th-century France, a blind teenager named Louis Braille thought of another way: using the sense of touch. Not with light, not with sound, but with raised dots. He invented a system that encodes letters to be read with the fingertips—Braille. This system is one of the earliest binary encoding systems used on a large scale in human history. In 1824, long before the birth of the electronic computer, a blind teenager had already encoded the entire alphabet using 0s and 1s.

3.1 The Dot Matrix: The Code Space of 6 Dots

Braille uses a rectangular matrix of 2 columns by 3 rows of dots to represent a single character. Each position can be either raised or flat—6 positions in total. Each position is a bit—raised is 1, flat is 0.

6 dots, each with two states. 2^6 = 64 possible combinations. Enough to cover 26 letters, numbers, punctuation marks, and some special abbreviations.

Letters A through J only use the top four dots. A is a single raised dot in the top-left corner, B is top-left plus top-right, C is top-left plus middle-left—and so on. Letters K through T build upon A through J by adding a raised dot in the bottom-left corner. Letters U through Z add yet another raised dot.

Every single Braille character is a 6-bit binary number. Braille is not metaphorically "like" binary—it **is** binary.

Light, sound, raised dots—the medium changes, but the logic remains entirely the same. Chapter 1 encoded using flashlight light, Chapter 2 encoded using multiple flashlights flashing simultaneously, Chapter 3 encodes using raised dots. Light can vanish, sound can dissipate, but raised dots can remain on paper and be touched repeatedly. The reason Braille has endured for two hundred years is that it chose a persistent and universal physical medium: the sense of touch. The essence of encoding is not the physical medium, but the rules. The physics can change endlessly, but the logic of a shared set of rules between sender and receiver never changes.

3.2 Why 6 Dots, and Not 8, 10, or 64?

This is a balance between physical constraints and encoding needs. The area a fingertip can perceive simultaneously at one time is limited; a 2×3 dot matrix is just the right size to cover the area of a single fingertip. More dots would require moving the finger, reducing reading speed. 64 combinations are enough to cover basic letters and punctuation. If more symbols are needed, they can be expanded by adding a second 2×3 dot matrix—this is the logic behind "Grade 2 Braille." This is not increasing the number of dots, but layering contextual rules onto the existing structure: the same dot matrix can represent different meanings in different contexts.

This is entirely consistent with the "encoding layers" found in computers. The same 8-bit byte, under ASCII encoding, is a character; under integer encoding, it is a number; under machine instruction encoding, it is an operation code. The physics remains unchanged, but the semantic layer differs.

3.3 The Tactile Channel: Low Bandwidth, But Persistent

Braille is a low-bandwidth channel—touching with a fingertip takes time, and reading speed is far slower than using sight. But it is persistent storage: once printed on paper, the raised dots can be preserved for decades and touched repeatedly. This is consistent with the principle of ROM (Read-Only Memory) in a computer—write once, read many times.

Raised dots on paper, after repeated touching, will wear down and become flat, leading to misreading. This is the same type of problem as "bit flipping" in computer memory—physical states change over time due to environment and wear. Braille has no built-in error correction mechanism,

but computers do: parity checks, ECC memory, CRC check codes—all of these exist to solve the problem of "unstable physical states."

Braille, once printed on paper, is fixed (Read-Only Memory). The next stop is the journey towards the relay—a switch that can repeatedly change state, can be automatically controlled, and can be "pressed" by an electrical signal instead of a fingertip. From touch to electromagnetism, from manual to automatic.

Chapter 4: Anatomy of a Flashlight

The first three chapters all discussed the logic of encoding—using light to represent information, using multiple flashlights to represent more information, using raised dots to represent letters. But all of this rests on one premise: the flashlight has to be able to turn on. If the flashlight doesn't light up, none of the stories that follow can exist.

So this chapter does not discuss encoding. Open up a flashlight and see what is actually inside.

4.1 Four Parts, One Closed Loop

The most ordinary flashlight, when taken apart, has only four components: a battery, wires, a switch, and a light bulb. Everything that follows—relays, logic gates, adders, flip-flops, CPUs, memory—are all just permutations and combinations of these four parts.

The battery, through a chemical reaction, gathers positive charges at its positive terminal and negative charges at its negative terminal. When the switch is closed and the circuit is connected, electrons set off from the negative terminal, flow along the wire, pass through the light bulb, and return to the positive terminal. The bulb lights up because the electrons, while flowing through the filament, collide with the filament atoms, causing the filament to heat up and glow. **Light is a byproduct of electrons colliding with atoms.**

A circuit must be a closed loop. If the switch is open, there is a gap in the circuit, electrons cannot cross it, the current stops, and the bulb goes out. The switch itself does not generate electricity, nor does it consume it; it only decides "whether or not to let the current pass through." This is the most primitive control element, but it contains a fundamental logic: using one physical quantity to control another physical quantity. Every subsequent chapter—relays, transistors, logic gates—are all extensions of this principle.

Imagine an electrical circuit as a closed system of water pipes. The battery is the water pump, the wires are the pipes, the switch is the valve, and the light bulb is the waterwheel. Water flows past the waterwheel, pushing it to turn; the water itself does not vanish, only its kinetic energy is consumed, and it continues to flow back to the pump. Electric current is the same—the electrons do not vanish, only their energy is consumed by the light bulb.

4.2 Voltage, Current, and Resistance: A Balancing Act

Circuit design is, at its core, about balancing three quantities. Voltage is the driving force that pushes electrons to flow. Current is the number of electrons flowing through a wire per unit of time. Resistance is the degree to which the flow of current is impeded. Ohm's Law states: Current equals Voltage divided by Resistance.

The flashlight bulb can light up because the battery provides sufficient voltage, the wire's resistance is low enough, and the filament's resistance is just right—the current flowing through the filament generates heat and light. If the wire is too thin (resistance is too high), the current is too low, and the bulb won't light. If the filament is too thick (resistance is too low), the current is too high, and the bulb will instantly burn out.

4.3 Why a Flashlight Isn't Perfect

A flashlight has its physical limits. Flicking a switch by hand can happen at most a few times per second, but a computer needs to switch on and off billions of times per second. A manual switch is completely unable to meet the speed requirement—Chapter 6 will replace the finger with an electromagnet, using a relay to achieve automatic switching.

The longer the wire, the higher the resistance, and the more severe the signal attenuation. A flashlight's wires are only a dozen or so centimeters long, but a submarine internet cable requires a repeater every few dozen kilometers, essentially "boosting" the signal to compensate for the attenuation caused by the wire's resistance. This is the problem Chapter 5 will solve.

While the light bulb emits light, it also generates a large amount of heat. Computer chips have the exact same problem—billions of transistors packed onto a silicon chip the size of a fingernail, each one generating heat. The fan, heatsink, and even water-cooling system inside your computer are all solving this heat dissipation problem that started with the flashlight bulb.

This chapter is the cornerstone of the physics portion of the entire book. Four parts—a battery, wires, a switch, and a light bulb—constitute the most primitive physical model of computer hardware. The following chapters will continue along this path: extending the wire (Chapter 5), replacing the finger with a relay (Chapter 6), assembling relays into logic gates (Chapter 10), assembling logic gates into a CPU (Chapter 17).

But the root is still this flashlight. It lights up.

Chapter 5: Seeing Around Corners

In Chapter 4, the two kids had windows directly facing each other. The flashlight's beam crossed the street, a straight-line distance of less than ten meters. But what if you wanted to talk to someone in the next room? Someone downstairs? Someone in another city?

Distance is the first enemy of communication.

5.1 Longer Wires, Longer Worries

The most direct solution: take apart the flashlight, and connect a longer wire between the battery and the bulb. Put the switch and the bulb in one room, and the battery in another. Press the switch, and the current flows along the long wire to the bulb, then back.

Completely feasible. But the longer the wire, the higher the resistance. With higher resistance, some of the voltage is consumed by the wire itself before reaching the bulb, making the bulb dimmer. Electrons travel very fast through the wire, but the rising and falling edges of a digital signal become blurred over a long wire—it takes longer for the signal to switch from one state to another. The longer the wire, the more severe the delay.

The law of resistance is simple: resistance is proportional to the length of the wire, and inversely proportional to its cross-sectional area. To make long-distance transmission viable, there are two paths—make the wire thicker, or increase the voltage. Thicker wire is too costly; the real-world solution is higher voltage. Power plants transmit electricity at hundreds of thousands of volts, then step it down near the end user, precisely to reduce losses during long-distance transmission.

But electricity has one advantage that light does not: the wire can bend. Light can only travel in a straight line, stopping when it hits a wall. Electricity can flow along a curved wire around any obstacle. Windows don't need to face each other; as long as a wire connects the two ends, communication can happen.

5.2 The Telegraph: Humanity Sends Information a Thousand Miles for the First Time

Extend the wire to dozens of kilometers, and you have a telegraph machine. It needs no fiber optics, no radio, no satellite—just a wire, a battery, and an electromagnet. The wire traverses fields, mountains, and seabeds, sending a signal a thousand miles away.

Morse code is the language of the telegraph. It encodes letters using two kinds of signals (dots and dashes), and the logic is exactly the same as the "long flash" and "short flash" of a flashlight. But Morse code is not purely binary—it has five symbols: dot, dash, inter-dot gap, inter-letter gap, and inter-word gap. It is essentially a variable-length encoding: common letters get short codes (E is the shortest, a single dot), while rare letters get long codes. This design philosophy of "the higher the frequency, the shorter the code" is in the same lineage as modern compression algorithms.

The simplest telegraph system needs only a pair of wires and two telegraph sets. When end A presses the key, the electromagnet at end B engages, making a "click" sound. A short press is a dot, a long press is a dash.

But the wire is shared. If both end A and end B press their keys at the same time, the currents will conflict, and neither side will receive the correct signal. This is the "collision" problem on a computer bus—modern bus protocols have dedicated arbitration mechanisms, but the most primitive telegraph system already exposed this fundamental contradiction: a shared channel requires arbitration.

5.3 Three Engineering Challenges Brought by Distance

A signal attenuates after traveling every certain distance. On long-distance telegraph lines, a repeater station is needed every few dozen kilometers—upon receiving the weak signal, it does not interpret the content, but simply retransmits the signal "as is" using a fresh battery and electromagnet. This is the prototype of the amplifier. The repeaters in modern submarine fiber optic cables and the transponders in satellite communications all follow the same logic: before the signal attenuates beyond recognition, "shout" it once more.

Two parallel long wires will interfere with each other—a signal change on one wire can induce a weak current on the other wire. This is "crosstalk." The solution from the telegraph era was to twist the two wires together, so that external interference induces currents of opposite polarity on the two wires, canceling each other out. This is the principle behind twisted-pair cabling, which is still used in Ethernet cables today.

The laying of the transatlantic telegraph cable in the 19th century was one of the most insane projects in the history of human engineering. The cable endured immense water pressure in the deep sea, and if it broke, specialized ships were needed to salvage and repair it. After the first transatlantic cable was activated, it failed within just a few months due to insulation quality issues. But its successful operation proved one thing: distance, though it brings a cost, is not an insurmountable barrier. With enough engineering resources, a signal can reach any corner of the Earth.

This chapter is the spatial extension of the Chapter 4 circuit. The wire is no longer a dozen or so centimeters, but dozens or thousands of kilometers. Distance brings attenuation, delay, conflict, and cost—but also gives rise to solutions like repeaters, twisted-pair cables, and submarine cables. And the heart of the telegraph machine—that device which uses an electromagnet instead of a finger to press a switch—is the protagonist of the next chapter. It's called the relay.

Chapter 6: Telegraphs and Relays

Chapter 5 solved the problem of "extending the wire"—but just making the wire longer is not enough. After traveling a hundred kilometers, the current is too weak to light anything. How do you make a weak signal strong again? And there's another problem: the kid in Chapter 1 could press the flashlight manually, but if Morse code needs to be continuously sent along a transcontinental line, who will press it for him?

The answer to both of these questions is the same thing: the relay.

6.1 The Electromagnet: Using Electricity to Control Magnetism

When a current passes through a wire, a magnetic field is generated around it. Coil the wire, and the magnetic field becomes concentrated inside the coil. Insert an iron core into the middle of the coil, and the magnetic field will magnetize the core, turning it into a much stronger electromagnet. When current flows, the iron core becomes a magnet; when the current stops, the magnetism vanishes. The essence of an electromagnet is using electric current to control magnetic force.

A relay binds an electromagnet and a switch together. When the coil is energized, the iron core generates magnetic force, attracting a nearby armature. The armature moves, connecting or disconnecting contacts on another circuit. The input circuit and output circuit are completely isolated electrically—there is no physical connection, only magnetic coupling.

A relay is a switch controlled by electric current. The input is the presence or absence of current; the output is the making or breaking of another circuit. A finger does not need to touch the switch; it only needs to connect or disconnect the coil's current from a distance, and the relay will automatically complete the switching action. This is the physical origin of "automation."

6.2 Relay Amplification: Shouting a Weak Signal Loud Again

In a telegraph system, the relay takes on an even more important role: relay amplification.

After a telegraph signal has traveled several hundred kilometers along a wire, the current is too weak to drive any device. But just before the signal completely vanishes, connect this weak signal to the coil of a relay—the coil only needs a very small current to attract the armature. Once the armature is attracted, it closes a local, brand-new, high-current circuit. The signal strength on this new circuit is restored to its original level and continues to propagate along the next segment of the line.

This is signal amplification: using a weak input signal to control a strong output signal. Place a relay at intervals along the telegraph line, and the signal can cross an entire continent. The transistors in modern digital chips are also essentially switches that perform amplification—using a tiny gate voltage to control a much larger source-drain current.

6.3 The Complete Telegraph System

At the sending end, the key is pressed, and the battery's current flows along the wire to the receiving end. At the receiving end, the relay coil is energized, the armature is attracted, causing the sounder to strike once—this is a "click." A short press is a dot, a long press is a dash. Morse code travels thousands of miles along the wire in this way.

Relay contacts have two basic configurations. Normally-open contacts: the circuit is broken when the coil is not energized, and connected when energized. Normally-closed contacts: the circuit is connected when the coil is not energized, and broken when energized. A single relay can have both types of contacts simultaneously—the combination of these two contact types is the physical foundation for all subsequent logic gates.

Relay contacts can be connected in series, in parallel, or inversely. Connect the contacts of two relays in series, and the circuit is only complete if both are attracted—this is the prototype of the "AND gate." Connect the contacts of two relays in parallel, and the circuit is complete if either one is attracted—this is the prototype of the "OR gate." Use a normally-closed contact instead of a normally-open one, and the circuit breaks when attracted, and connects when not attracted—this is the prototype of the "NOT gate."

Chapter 10 will formally discuss logic gates, Chapter 11 will use logic gates to build an adder, and Chapter 17 will assemble an adder, memory, and controller into a CPU. But all of this originates from a simple relay—coil, iron core, armature, contacts. The entire computer is just a permutation and combination of a pile of relays.

6.4 The Relay Isn't Perfect

A relay relies on mechanical movement—the armature attracting and springing back. Mechanical movement has inertia and cannot be too fast; a single relay can switch at most dozens to hundreds of times per second. When the armature is attracted, it will also bounce back and forth between the contacts several times before stabilizing. This "bounce," although only a few milliseconds long, is enough to misread a single switching action as several distinct ones. Telegraph operators relied on their ears to judge—a clear "click" sound is a valid signal; a blurred, buzzing sound is bounce. The human ear served as a debounce filter.

Every time a relay contact opens or closes, a tiny arc is generated at the instant of contact, eroding the contact surface. Over time, this causes poor contact. Telephone exchanges in the early 20th century used relays to connect lines, and a routine part of maintenance work involved regularly cleaning or replacing contacts.

This is why the relay was eventually replaced by the transistor. The principle of the transistor is also "using a tiny signal to control a large signal," but it has no mechanical movement, relying purely on the movement of electrons within a semiconductor material. Its switching speed can reach billions of times per second. But logically, the transistor is the descendant of the relay—simply replacing the armature with electrons, and the coil with a gate.

This chapter is the logical origin of the book's hardware. The relay can both amplify a signal and automatically control a circuit, and it can be connected in series, in parallel, or inversely to form logic gates. The following chapters will continue along this path—binary numbers (Chapters 7-9), logic gates (Chapter 10), adders (Chapter 11), flip-flops (Chapter 14), the CPU (Chapter 17). All of these things, at their root, are a switch that controls current with current. It's called the relay.

Chapter 7: Our Ten Digits

The previous six chapters discussed hardware—flashlights, wires, relays. But hardware can only process two states: "on" and "off." Before assembling relays into a computer, a more fundamental problem needs to be solved first: What is number itself? Why are we accustomed to carrying over after counting to ten? Why must computers use binary?

7.1 Decimal: An Anatomical Accident

Humans use the decimal system not because it is a mathematical necessity, but because we are born carrying a counting tool: ten fingers. Counting on one's fingers is the common starting point of almost all ancient civilizations. After counting all ten fingers, with no fingers left to count, a mark is made on the ground—this is the carry—and then counting starts over with the fingers. Decimal is not a choice of mathematics, but a choice of anatomy.

The essence of any numeral system is the same: a fixed set of symbols, used repeatedly in cycles, with position indicating a higher order of magnitude. The decimal system has ten symbols (0-9); after counting to 9, the symbols are exhausted, so a place is added to the left, and the rightmost place is reset to 0—this is "carry the one at ten." An arbitrary base-N numeral system has N symbols; after counting to N-1, the symbols are exhausted, and a carry occurs.

Any number can be expanded by place into a sum of the digit multiplied by the base raised to a power. This principle of positional weighting is the foundation for all numeral system conversions. No matter the numeral system, the result of a mathematical operation is equivalent—twelve plus thirteen always equals twenty-five, whether you represent this result in decimal, binary, or the Mayan vigesimal system.

A positional numeral system inherently requires a symbol meaning "there is nothing here." When a place has no value, a placeholder is needed to distinguish 35 from 305—this is the precursor to zero. Early Babylonian numerals used a space to represent an empty place, which later evolved into a dedicated symbol. Without a placeholder, it is impossible to distinguish between positions of different magnitudes.

7.2 History Has Seen More Than One Numeral System

Every civilization used its own body parts to count. Ancient Egypt, ancient China, ancient Greece, and ancient Rome all used decimal—ten fingers. Ancient Babylon used sexagesimal (base-60)—finger joints, 12 joints × 5 fingers = 60. This base-60 system still lingers in our timekeeping today: 1 hour = 60 minutes, 1 minute = 60 seconds, a circle has 360 degrees. The Maya used vigesimal (base-20)—fingers plus toes. Computers use binary—the two states of a switch.

No numeral system is more "correct" than another. They are simply different physical implementations of the same underlying principle.

7.3 Why Computers Chose Binary

When humans build calculating tools, the choice of numeral system directly determines the complexity of the physical implementation. A decimal mechanical computer requires a gear with ten teeth, each tooth corresponding to a single digit. The gear manufacturing requires high precision, and the carry mechanism is complex. Binary only requires two states: on or off, high voltage or low voltage, magnetized or not magnetized. The physical implementation of two states is far simpler than ten states—a relay is either attracted or released, a transistor is either conducting or cutoff.

Computers chose binary not because it is mathematically superior, but because it is physically the easiest to implement.

The cost of binary is the number of digits. The decimal number 1000, written in binary, is 1111101000—10 digits. Humans are not good at reading long strings of 0s and 1s. This is why the computing field introduced octal and hexadecimal—they are abbreviations for binary. Four binary digits correspond exactly to one hexadecimal digit, and conversion requires no calculation, only grouping. Hexadecimal is the bridge between the programmer and binary.

This chapter lays the groundwork for Chapter 8. Why binary is the natural language of switches (Chapter 8), how to perform addition and subtraction in binary (Chapter 9), how to implement binary addition using relays (Chapter 12), and how hexadecimal becomes the most compact, human-readable form of binary (Chapter 15).

But all of this begins with a single fact: we count to ten because we have ten fingers. Computers count to two because a switch has only two states. That is all.

Chapter 8: Alternatives to Ten

Chapter 7 made it clear: humans use the decimal system only because we have ten fingers, not because of any mathematical necessity. Since decimal is just one among many possible numeral systems, what is the simplest one?

Binary. It only needs two symbols—0 and 1—to represent any number. These two symbols happen to correspond perfectly to a relay being "open" or "closed," and a transistor being "cutoff" or "conducting."

8.1 Walking Down the Path of Decreasing Symbols

Decimal has ten symbols (0-9), carrying over at ten. Octal has eight symbols (0-7), carrying over at eight. Quaternary has four symbols (0-3), carrying over at four. Binary has two symbols (0 and 1), carrying over at two.

With each reduction in symbols, the moment when "symbols run out and a carry is needed" arrives sooner. Binary has the fewest symbols, so it carries over most frequently. Count 0, then 1, then the symbols run out, and a carry must happen, becoming 10—the number 10 in binary equals 2 in decimal. Each position in binary represents a power of 2. From right to left: the first place is 1, the second is 2, the third is 4, the fourth is 8.

Octal was used in early computers because every 3 binary digits correspond exactly to 1 octal digit, making conversion simple. Today, octal is almost only visible in Linux file permissions— `chmod 755`, each digit represents 3 binary bits (read, write, execute). `7=111=rwx`, `5=101=r-x`. This is the last surviving remnant of octal in modern computer systems.

8.2 Binary to Decimal, Decimal to Binary

A number in any numeral system can be expanded by place: multiply the value of each digit by its corresponding weight, then sum them all. The weight of each binary place is a power of 2—from right to left: 1, 2, 4, 8, 16. Multiplying the value of each place by its weight and summing yields the decimal result.

To convert decimal to binary, use the "divide by 2 and record the remainder" method. Repeatedly divide by 2, recording the remainder (0 or 1) each time, until the quotient is 0. Arrange the remainders from last to first, and that is the binary result.

8.3 The Biggest Drawback of Binary

Too many digits. The decimal number 1024, in binary, is 10000000000—11 digits. The human brain is not good at processing long strings of 0s and 1s. This is why Chapter 15 will introduce hexadecimal: 4 binary digits correspond exactly to 1 hexadecimal digit. A-F are not letters, they are numbers—A=10, B=11, C=12, D=13, E=14, F=15. Hexadecimal is a compressed representation of binary.

Binary can represent not only integers, but also fractions. The weights to the right of the decimal point are 0.5, 0.25, 0.125… halving each time. But some fractions that can be represented precisely in decimal become infinitely repeating fractions in binary—just as 1/3 = 0.333... is infinitely repeating in decimal. This is why 0.1 + 0.2 does not equal 0.3 in floating-point arithmetic. This is not a bug in the computer; it is a mathematical property of binary itself. Chapter 23 will detail the IEEE 754 standard for floating-point numbers.

This chapter is the foundation for binary operations and applications. Binary addition and subtraction (Chapter 9), implementing binary addition with relays (Chapter 12), hexadecimal as the most compact, human-readable form of binary (Chapter 15), and fixed-point and floating-point numbers (Chapter 23)—all begin here.

But everything begins with a single fact: two symbols, 0 and 1, can represent any number. Not because binary is the most elegant, but because two states are the easiest to implement. A relay is either attracted or released. A transistor is either conducting or cutoff. Computers chose binary not out of mathematical necessity, but out of physical necessity.

Chapter 9: The Bitwise Operations

Chapter 8 solved the problem of "how to represent numbers in binary." But just being able to write them is not enough; you must also be able to calculate with them. If binary cannot perform arithmetic, then it's just another way of writing numbers, with no engineering value whatsoever.

Binary addition is far simpler than decimal addition. Not just "a little simpler"—it is so simple there are only four rules. Hold out your hand and imagine two fingers. Each finger can only be raised or lowered (1 or 0). When both fingers are raised simultaneously (1 + 1), you find they are not enough—two fingers can only represent two things, but now you have three things to represent (1, 1, and the result, 2). So you lower both fingers (the sum bit returns to 0) and then take a coin out of your pocket and place it on the table—this is the carry.

1 + 1 = 0, carry 1. That is the entirety of binary addition.

9.1 Four Rules, Two Logic Gates

List all the possible combinations of those two fingers: $0 + 0 = 0$ carry 0, $0 + 1 = 1$ carry 0, $1 + 0 = 1$ carry 0, $1 + 1 = 0$ carry 1.

Observe the "sum" column: it is XOR—1 when the two inputs are different, 0 when they are the same. Observe the "carry" column: it is AND—1 only when both inputs are 1. This is no coincidence. This is the deepest beauty of binary addition: the sum bit is XOR, the carry bit is AND. Two logic operations can calculate the addition result of any two bits.

This means an adder can be built using relays. An XOR gate and an AND gate can each be implemented by connecting a few relays in series and parallel. Put them together—input two bits, output a sum bit and a carry bit—this is a half adder. Connect two half adders end to end to form a full adder. String eight full adders together, and let the carry ripple from right to left, and you have an 8-bit adder.

This chain—**Arithmetic → Logic → Switches**—is the single most critical logical thread running through the entire book *Code*. The four addition rules of this chapter are the very first link in that chain.

9.2 The Carry Ripple: The Speed Limit of an Adder

Multi-bit binary addition starts from the least significant bit on the far right, adding place by place, with the carry propagating to the left. This process is like dropping a pebble into a pond—the ripples spread outward from the point of impact, and each bit is a peak in that ripple. In a relay-based adder, each expansion of this ripple is the engagement time of a single relay. In the worst case, an 8-bit adder must wait for 8 relays to engage in sequence before the most significant bit can stabilize.

This is the fundamental bottleneck for the speed of an adder. Modern CPUs break this bottleneck using "carry lookahead" technology—instead of waiting bit by bit for the ripple to arrive, an extra layer of logic gates calculates the carry values for all bit positions simultaneously. The cost is more logic gates; the benefit is speed.

9.3 Subtraction: Turn It Into Adding a Negative

Binary subtraction on paper is exactly the same as decimal—when a place is too small to subtract from, "borrow 1 to become 2" from the higher place. But implementing borrowing with circuits is troublesome, requiring magnitude comparison and borrow propagation. The computer does not take this path.

It turns subtraction into addition. When calculating A - B, it turns B into a negative number and then adds it to A. How do you represent a negative number? Invert every bit of B, and then add 1. This operation is called "finding the two's complement." The complement of B, when added to B itself, results in an overflow back to zero—so the complement is the additive inverse of B. Adding the complement is equivalent to subtracting the original number.

This means a computer only needs a single adder to perform both addition and subtraction. One set of hardware, two operations. This is one of the most elegant designs in all of computer architecture.

9.4 Multiplication and Overflow

The rules of binary multiplication are even simpler: any number multiplied by 0 is 0, and multiplied by 1 is itself. Multi-bit multiplication is simply "left-shift + add." Shifting one place to the left is equivalent to appending a 0 at the end—this is the same logic as multiplying by 10 in decimal by appending a 0 at the end, except in binary, multiplying by 2 appends a 0. Binary multiplication is, at its heart, repeated shifting and adding—and both of these operations can be implemented with relays.

But all of these operations are bounded by the number of bits in the adder. An 8-bit adder can only represent values from 0 to 255 at most. When the result of adding two numbers exceeds this range, the carry from the most significant bit is lost—this is overflow. The computer uses a "carry flag" in the status register to record overflow, and a program can check this flag to decide on subsequent actions.

This chapter is the blueprint for Chapter 12. Four addition rules → Half Adder → Full Adder → Carry Chain → 8-bit Adder. The two's complement turns subtraction into addition. Bit-shifting turns multiplication into repeated addition.

And the starting point for all of this is the two fingers you extended. Raised is 1, lowered is 0. 1 + 1 = 0, carry 1. The relay engages, the armature drops, the contact closes. Current flows, and the next place receives a carry signal. Not a formula, but a relay. Not a truth table, but an electric current.

Chapter 10: Logic and Switches

Chapter 9 left behind a question: XOR and AND, these two "logical operations"—how can they be implemented using relays?

This chapter answers that question. It introduces three fundamental types of logic—AND, OR, NOT—and then proves that these three logics can be physically implemented using the series connection, parallel connection, and inverse connection of switches. This is the first time the entire book welds "logic" and "physics" together. In 1854, George Boole used mathematical symbols to describe the operational rules of "true" and "false." Sixty years later, a graduate student named Claude Shannon proved in his master's thesis that Boolean algebra could be implemented with relay circuits. This thesis has been called "the foundational work of the Information Age."

10.1 Series is AND, Parallel is OR

The logic rule of the AND gate: the output is true only if A AND B are simultaneously true. To implement this with switches: connect two switches in series. The power source goes out, passes through switch A, then through switch B, to the light bulb, and back to the power source. Only when both switches are closed can the current flow through. If either one is open, the circuit is broken, and the bulb goes out.

The logic rule of the OR gate: the output is true if either A OR B is true. To implement this with switches: connect two switches in parallel. The power source splits into two branches, one going through switch A to the bulb, the other going through switch B to the bulb. As long as one switch is closed, the current has a path to reach the bulb. Only when both are open does the bulb go out.

Series = AND, Parallel = OR. This is the translation dictionary between logic and circuits.

10.2 The NOT Gate: The Only Gate That "Flips"

The NOT gate is the most special. It has only one input, and its output is always the opposite of its input—input true, output false; input false, output true. To implement this with a relay: use a normally-closed contact. When the coil is not energized, the armature rests against the normally-closed contact, the circuit is complete, and the bulb lights up. The moment the coil is energized, the electromagnet pulls the armature away, the normally-closed contact opens, and the bulb goes out. Energized is input true, the bulb off is output false. De-energized is input false, the bulb on is output true.

The NOT gate is the only gate that can "flip" a signal. It is also the core component of the flip-flop (memory circuit) in the following chapters—two NOR gates feeding back into each other form a circuit that can "remember" a 0 or a 1.

10.3 NAND and NOR: You Only Need One Type of Gate

In actual circuit design, the NAND gate and the NOR gate are more commonly used than simple AND and OR gates. The reason is simple: using NAND gates alone, you can combine them to produce all three types of logic—AND, OR, and NOT. This means you only need one type of gate to build an entire digital system. Simple AND gates or OR gates cannot do this alone—they must be paired with a NOT gate to achieve all the logic. This is why many chips internally only use NAND gates: not because they are "better," but because they are "sufficient"—one type of gate, mass-produced, at the lowest cost.

10.4 From Relays to Transistors: The Physics Changed, the Logic Did Not

A relay relies on mechanical movement—the armature attracts and springs back, taking several milliseconds. A transistor relies on an electric field to control the flow of electrons in a semiconductor, taking several nanoseconds. The speed differs by a factor of a million. The logic of the

logic gates—AND, OR, NOT—has not changed at all. Only the physical implementation has changed, from "mechanical armature" to "semiconductor electric field."

Every time a relay contact opens or closes, a tiny arc erodes it, and over time, the contact becomes poor. A transistor has no contacts, no wear, and no springs to fatigue. But their mathematical essence is exactly the same: using an input to control an output, using the series and parallel connection of switches to implement logical operations.

This chapter is the weld point between logic and hardware. The standardized symbols for logic gates (Chapter 11), using logic gates to build an adder (Chapter 12), using two NOR gates to form a flip-flop (Chapter 14), assembling logic gates into a CPU (Chapter 17)—all begin with the series and parallel connections of this chapter.

Positive terminal of the power source. Switch A. Switch B. Light bulb. Negative terminal of the power source. In series, that's AND. In parallel, that's OR. This is the physical origin of logic. Not a formula, but a switch. Not a truth table, but an electric current.

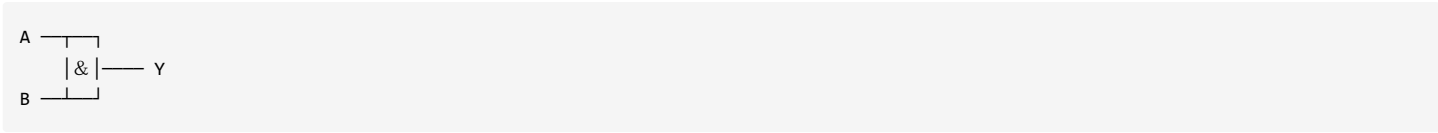
Chapter 11: Gates

Chapter 10 used relays to implement AND gates, OR gates, and NOT gates. But every single gate contained a pile of parts—a coil, an iron core, an armature, contacts, a spring. If you were to build an adder, you would need dozens of gates, and if every gate were drawn as a relay, the circuit diagram would become too complex to read.

This chapter does just one thing: give each type of logic gate a standardized symbol, and then forget about the relays inside it. From now on, we only need to care about "what are the inputs and outputs of this gate," and no longer need to care about "how the armature inside this gate engages." This is the very first layer of genuine abstraction in the history of computer engineering.

11.1 The AND Gate

The logic rule of the AND gate is: the output is 1 only if A AND B are simultaneously 1; otherwise, the output is 0. Its symbol is a D-shape (a semicircle on the left, straight edge on the right):

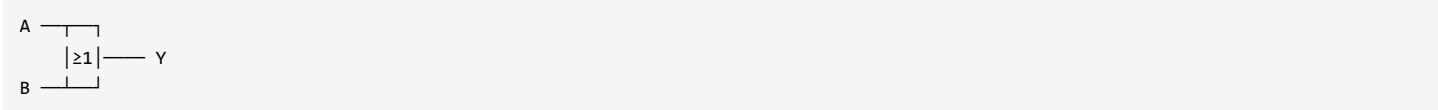


A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Two switches connected in series is an AND gate.

11.2 The OR Gate

The logic rule of the OR gate is: the output is 1 if either A OR B is 1; only if both are 0 is the output 0. Its symbol is an arc shape (curved on both the left and right sides, coming to a point on the right):



```
A B | Y
0 0 | 0
0 1 | 1
1 0 | 1
1 1 | 1
```

Two switches connected in parallel is an OR gate.

11.3 The NOT Gate (Inverter)

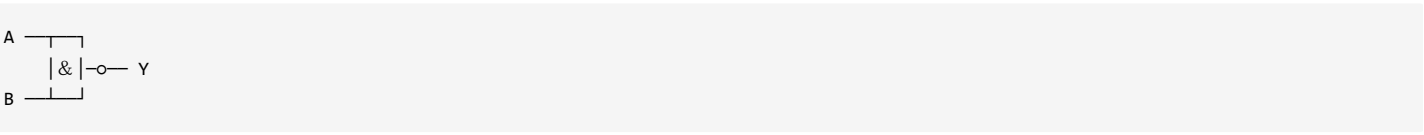
The logic rule of the NOT gate is: if the input is 1, the output is 0; if the input is 0, the output is 1. It has only one input and one output, making it the most special of all the gates. Its symbol is a triangle plus a small circle—the triangle represents "amplification" (driving), and the small circle represents "negation":


```
A | Y
0 | 1
1 | 0
```

A relay's normally-closed contact is a NOT gate. When the coil is not energized, the armature rests against the normally-closed contact, the circuit is complete; when the coil is energized, the armature is pulled away, the circuit is broken. Input energized = output broken, input de-energized = output complete.

11.4 The NAND Gate

A NAND gate is an AND gate followed by a NOT gate. Logically, it is "the result of the AND operation, then negated." Its symbol is the D-shape of an AND gate with a small circle added at the output:

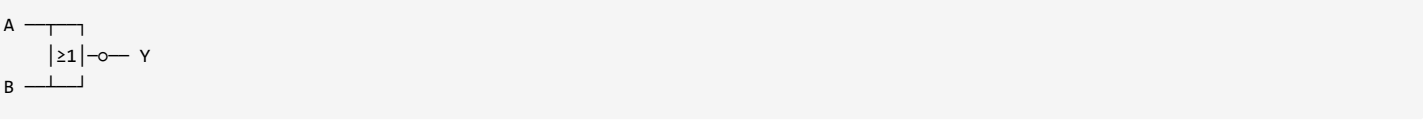



```
A B | Y
0 0 | 1
0 1 | 1
1 0 | 1
1 1 | 0
```

The NAND gate is **functionally complete**—using NAND gates alone, you can combine them to produce all three types of fundamental logic: AND, OR, and NOT. This means you only need one type of gate to build an entire digital system. Modern chips often use only NAND gates internally: not because they are "better," but because they are "sufficient"—mass-producing one type of gate yields the lowest cost.

11.5 The NOR Gate

A NOR gate is an OR gate followed by a NOT gate. Logically, it is "the result of the OR operation, then negated." Its symbol is the arc shape of an OR gate with a small circle added at the output:

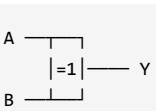


```
A B | Y
0 0 | 1
0 1 | 0
1 0 | 0
1 1 | 0
```

The NOR gate is likewise **functionally complete**. The NAND gate and the NOR gate are the two "universal building blocks" of digital circuit design.

11.6 The XOR Gate

The XOR gate already appeared in Chapter 9—the Sum bit equals A XOR B. Its logic rule is: the output is 1 if A and B are different; if they are the same, the output is 0. Its symbol is the arc shape of an OR gate with an additional curved line added at the input:




```
A B | Y
0 0 | 0
0 1 | 1
1 0 | 1
1 1 | 0
```

The XOR gate is the core component of an adder—without considering the carry-in, the Sum bit equals A XOR B.

11.7 Abstraction: No Longer Needing to Care How a Relay Engages

Starting from this chapter, we will no longer draw relay coils and contacts. Draw an AND gate as a D-shape, an OR gate as an arc shape, a NOT gate as a triangle plus a circle. As for how the relays inside these symbols work, we no longer care.

The essence of abstraction is encapsulating complexity and exposing an interface. A gate's symbol is the interface—inputs, output, functional description. The relay circuit inside the gate is an implementation detail. Moving upward, an adder encapsulates logic gates, a CPU encapsulates adders and registers, an operating system encapsulates the CPU and memory, and a programming language encapsulates the operating system and machine code. Each layer encapsulates the complexity of the layer beneath it and exposes only a simple interface to the layer above. And the starting point of all of this is that tiny D-shaped symbol.

11.8 Every Gate Steals Time

A signal does not pass through a gate instantaneously. The armature of a relay takes several milliseconds to engage; the gate of a transistor takes several nanoseconds to charge. This is "propagation delay." One gate delays by a few nanoseconds; a million gates connected in series can accumulate a delay into the millisecond range. This imposes a hard constraint on the clock frequency—the clock cycle cannot be shorter than the slowest logic path, because the signal must arrive and stabilize before the next clock sampling point.

There is also a limit on how many subsequent gates a single gate can drive—excessive fan-out leads to signal attenuation and timing skew.

This chapter is the standardized interface definition for logic gates. The adder (Chapter 12), the flip-flop (Chapter 14), the ALU (Chapter 17)—every subsequent circuit will be drawn using these standard symbols. But from now on, we only need to look at the D-shaped symbol and the arc-shaped symbol, and no longer need to think about relays. This is the power of abstraction. It lets you forget the details and focus on the larger structure. The very first layer of abstraction starts with this D-shaped symbol.

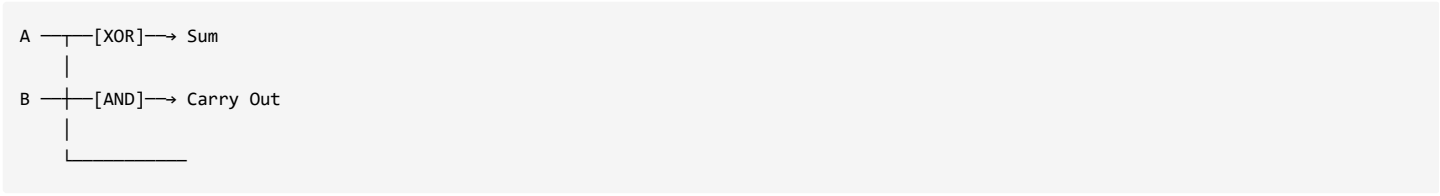
Chapter 12: A Binary Adding Machine

Chapter 9 proved that binary addition only requires two logic operations: the sum bit is XOR, the carry is AND. Chapter 10 proved that these two logic operations can be implemented using relays. Chapter 11 encapsulated the relay circuits into standardized gate symbols.

Now, all the parts are ready. This chapter will assemble a binary adding machine that can actually run. This is the first time the entire book crosses from "logic" to "computation"—before this, relays only controlled the making and breaking of circuits; after this, relays will calculate 1+1=2. This is the very first muscle of the computer.

12.1 The Half Adder: Can Only Add Two Bits

The half adder is the simplest addition circuit. It inputs two bits, A and B, and outputs Sum and Carry. An XOR gate outputs Sum; an AND gate outputs Carry.



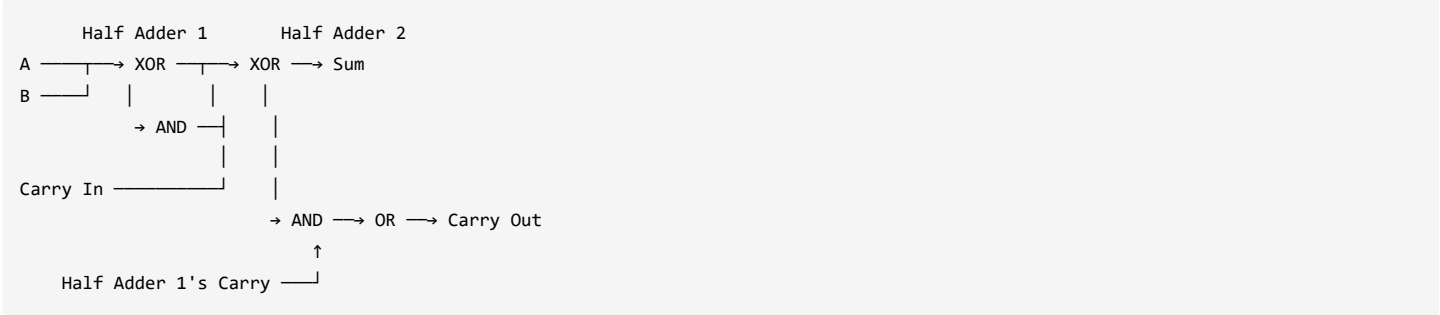
A	B	Sum	Carry Out
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

1+1=2, written in binary as 10—Sum=0, Carry=1. Perfectly correct. But the half adder has a fatal flaw: it has no "Carry In." It can only add two bits, not three. In multi-bit addition, each place must be able to receive a carry from the previous place—a half adder cannot do this.

12.2 The Full Adder: Adding Three Bits

The full adder has three inputs: A, B, and Carry In (the carry from the lower place), and two outputs: Sum and Carry Out (the carry going to the higher place).

The logic is broken into two steps: first, use one half adder to calculate A+B, obtaining an intermediate Sum' and intermediate Carry'. Then use a second half adder to calculate Sum' + Carry In, obtaining the final Sum and an intermediate Carry". If either of the two carries generated—Carry' or Carry"—is 1, then Carry Out is 1—use an OR gate to merge them.



Two half adders assembled into one full adder. The full adder can process A + B + Carry In—this is everything needed for each single place in multi-bit addition.

12.3 The Adder: Eight Full Adders Connected End to End

Connect eight full adders in series. The Carry In of the least significant bit is connected to ground (fixed at 0), and the Carry Out of each stage is connected to the Carry In of the next stage. A and B each provide 8-bit inputs, outputting an 8-bit Sum and a final Carry Out.

The Carry In of the least significant full adder is fixed at 0—because there is no carry from a lower place. If the Carry Out of the most significant full adder is 1, it means the result cannot be stored in 8 bits—this is overflow.

The carry propagates from the least significant bit to the most significant bit, place by place, like a ripple. In a relay-based adder, the delay of each full adder stage is the engagement time of one relay. In the worst case, an 8-bit adder must wait for the engagement time of 8 relays. This is the fundamental bottleneck for the speed of an adder. Modern CPUs use "carry lookahead adders"—using extra logic gates to calculate all carries simultaneously, instead of waiting for the ripple to arrive bit by bit. The cost is more logic gates; the benefit is speed.

12.4 From 8 Bits to 64 Bits

When the bit width expands to 16 bits, 32 bits, 64 bits, the principle remains completely unchanged—just use more full adders in series. The 64-bit adder inside a modern CPU is, in essence, 64 full adders connected end to end. From the relay era to the integrated circuit era, the logic of the adder has not changed; only the physical implementation has changed. A relay-based adder is the size of a shoebox; an integrated circuit adder is only a few hundred nanometers wide.

12.5 The Adder → The ALU

The adder is the core of the ALU (Arithmetic Logic Unit). An ALU can not only perform addition, but also subtraction (by using the two's complement mechanism, turning subtraction into addition), AND, OR, NOT, XOR, and shifting. A complete ALU consists of one adder plus control logic—the control logic selects which specific operation to execute based on the instruction. This is precisely the focus of Chapter 17.

This chapter is the first milestone in computing hardware. From logic gates to half adder, from half adder to full adder, from full adder to an 8-bit adder—for the first time, a relay is not just controlling the making and breaking of a circuit, but doing arithmetic. 1+1=0, carry 1. Eight full adders connected end to end, current flowing through, and the output is the sum of two binary numbers. The very first muscle of the computer begins to contract.

Chapter 13: But What About Subtraction?

Chapter 12 built an 8-bit adder. It can add. But a computer cannot only add; it must also be able to subtract. The most direct idea is to build another subtractor—use a different set of logic gates to assemble a circuit dedicated to subtraction. But a subtractor is more complex than an adder: it needs to compare magnitudes, handle borrowing, and deal with negative numbers.

Engineering has a smarter solution: don't build two sets of circuits. Just reuse the existing adder and use a mathematical trick to turn subtraction into addition.

13.1 The Complement: Turning Subtraction Into "Adding a Negative"

In the decimal system, "subtract 8" has the same result as "add 2 then subtract 10." 2 and 8 are mutual "complements to 10"—the complement is the number needed to "make ten."

The same logic applies in binary. "Subtract 1" equals "add 1 then subtract 2." The binary complement of 1 is obtained by inverting all its bits and then adding one. The two's complement of any number equals inverting all its bits and then adding one. For an 8-bit number (0~255), its two's complement equals 256 minus that number. The binary representation of this is exactly the same as the result of "invert then add one." The most significant bit naturally becomes the sign bit—0 for positive, 1 for negative. The two's complement of a positive number is the number itself; the two's complement of a negative number is the result of inverting all its bits and adding one.

The complete process for calculating A - B: Step one, invert all bits of B (0 becomes 1, 1 becomes 0). In the circuit, each line passes through a NOT gate. Step two, add one to the inverted result. Cleverly utilize the Carry In of the full adder—normally connected to ground (0), but set to 1 when performing subtraction, automatically completing the "add one." Step three, use the same adder to add A and the two's complement of B. The internal structure of the adder remains entirely unchanged; only the wiring of the input end is changed.

```
Addition mode: Input B remains unchanged, Carry In = 0
Subtraction mode: Input B is inverted (passes through NOT gates), Carry In = 1
```

One set of adder hardware, two operations. Hardware cost is nearly unchanged, functionality is doubled.

13.2 Why Two's Complement Won

Early computers experimented with "sign-magnitude" (the most significant bit represents the sign, the remaining bits represent the magnitude) and "ones' complement" (inverting all bits represents a negative number). Both schemes had fatal flaws: sign-magnitude had two zeros—positive zero (00000000) and negative zero (10000000)—and the adder could not process them directly; ones' complement likewise had two zeros—positive zero (00000000) and negative zero (11111111)—and the adder required "end-around carry" to process them.

Two's complement has only a single zero (00000000), the adder can process it directly, and no additional judgment logic is needed. All modern computers use two's complement to represent integers. This is the optimal solution in engineering.

13.3 Overflow: When the Result Can't Fit

When performing subtraction using two's complement, if the result exceeds the representable range of the bit width, the carry from the most significant bit is lost—this is overflow. But overflow in two's complement has an important property: after discarding the carry from the most significant bit, the value in the lower bits happens to be the correct result of the modular arithmetic. The program needs to judge on its own whether the result has overflowed, by checking the sign bit or the carry flag.

In a modern CPU, the switching between addition and subtraction is achieved through a single control line: 0 = addition, 1 = subtraction. This control line determines whether each bit of B passes through a NOT gate, and is simultaneously connected directly to the Carry In of the least significant bit of the adder. The delay of a NOT gate is far less than the delay of an adder, so subtraction and addition execute at the same speed—in a modern pipelined CPU, both are completed in a single clock cycle.

This chapter is the convergence point for arithmetic logic. One set of adder hardware, through the two's complement mechanism, can simultaneously perform addition and subtraction. Chapter 14 will introduce the flip-flop—the adder is combinational logic, its output depends only on the current inputs; the flip-flop is sequential logic, capable of remembering previous states. Chapter 17 will use the addition and subtraction logic of this chapter to design a complete ALU. Chapter 23 will extend integer addition and subtraction to floating-point numbers.

But the core idea is already set in stone in this chapter: don't build two sets of circuits. Use mathematics to simplify the hardware. This is the most elegant design philosophy in all of computer architecture—using cleverness in place of cost.

Chapter 14: Feedback and Flip-Flops

All the circuits built in the previous thirteen chapters—logic gates, adders, subtractors—share a common characteristic: the output depends only on the current inputs. If the input changes, the output immediately follows. This type of circuit is called "combinational logic." It cannot "remember" what just happened.

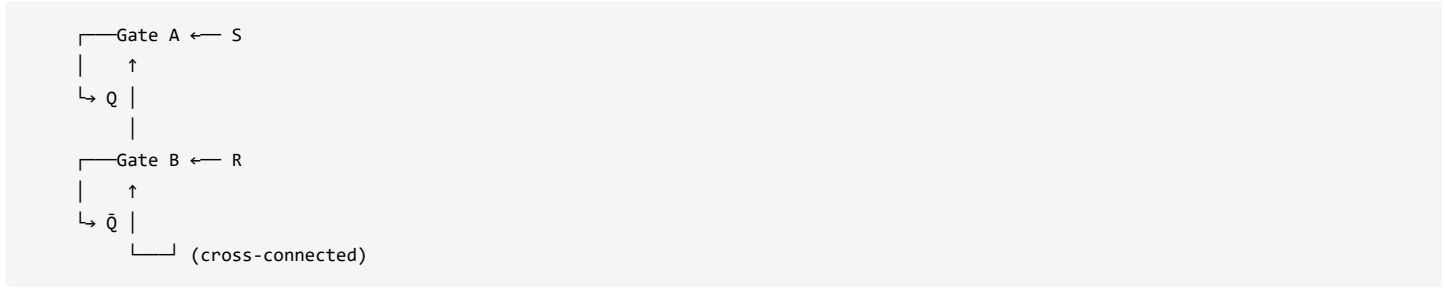
But a computer needs memory. You need to store numbers, instructions, and calculation results. You need a circuit that can "hold a state." This chapter introduces a revolutionary concept: feedback—connecting the output of a logic gate back to its input. This simple connection gives a circuit the ability to "remember" for the very first time.

14.1 Feedback: Connecting the Output Back to Itself

Connect the output of a NOT gate back to its own input. The output of a NOT gate is always the inverse of its input—once connected back to itself, the input is no longer controlled by the outside world, but by its own output. The circuit will oscillate wildly between 0 and 1, with the oscillation frequency depending on the propagation delay of the NOT gate. This "ring oscillator," although unable to reliably store information, reveals the essence of feedback: a circuit can possess its own internal state and does not have to completely obey external inputs.

14.2 The RS Flip-Flop: Two NOR Gates Locked Onto Each Other

Connect two NOR gates crosswise to each other—the output of gate A connects to one input of gate B, and the output of gate B connects to one input of gate A. The two remaining free ends serve as the S (Set) and R (Reset) inputs, respectively.



S=0 and R=0: The circuit remains unchanged. If Q is 0, it stays 0; if it is 1, it stays 1. It "remembers" the previous state. S=1 and R=0: Q is set to 0, $\bar{Q}$=1. After S is removed, the circuit holds Q=0—a 1 has been stored. S=0 and R=1: Q is set to 1, $\bar{Q}$=0. After R is removed, it holds Q=1—a 0 has been stored. The only forbidden combination is S=R=1: both NOR gate outputs are forced to 0, and the circuit enters an invalid state.

The logic of a NOR gate is: if at least one input is 1, the output is 0. Only when both inputs are 0 is the output 1. When S=1, no matter what the other input of gate A is, its output Q is 0. This 0 feeds back to gate B. Both inputs of gate B are 0 (R=0, Q=0), so its output $\bar{Q}$ is 1. $\bar{Q}$=1 feeds back to gate A. Even if S has been removed by now and gone back to 0, gate A's inputs are 0 and 1—at least one is 1, so the output Q remains 0. The circuit is latched.

The secret of memory is not the material, but the structure. An adder has no feedback; a flip-flop has feedback. Current circulating in a loop between two NOR gates can "remember" what just happened. This is analogous to an autocatalytic reaction in chemistry—the product is simultaneously the catalyst; once the reaction starts, it is self-sustaining. A flip-flop is an autocatalytic reaction in a circuit.

14.3 The D Flip-Flop: Adding a Clock Line

The RS flip-flop needs two input terminals, S and R, to control writing—one writes 1, the other writes 0. This is somewhat inconvenient to use. The D flip-flop merges the two inputs into a single D (Data) input, and adds a Clock input. When D=1 and a clock pulse arrives, Q becomes 1. When D=0 and a clock pulse arrives, Q becomes 0. When the clock is not pulsing, D can change as much as it likes; Q simply ignores it.

A D flip-flop is typically built as "edge-triggered"—it samples the data only at the very instant of the clock's rising edge (or falling edge), and completely ignores input changes at all other times. This is equivalent to installing a time gate on the data path, precisely controlling "when data is allowed in" down to the nanosecond level.

Your CPU registers, caches, and memory controllers are all constructed from billions of D flip-flops. The clock signal is like a heartbeat—with every heartbeat, all flip-flops simultaneously sample their inputs and simultaneously update their states. This is the meaning of a "synchronous circuit."

14.4 Metastability: The State Flip-Flops Fear Most

The illegal input S=R=1 is not just a theoretical forbidden zone; in reality, it can cause serious problems: both NOR gate outputs are forced to 0, and the circuit enters a floating state that is neither 0 nor 1. This state might last an extremely short time before randomly flipping to one side, or it might persist for too long, causing subsequent circuits to read an incorrect value. Modern digital systems avoid metastability through rigorous timing design: ensuring sufficient setup time and hold time between the clock and data signals.

On a large chip, the clock signal needs to travel through wires to millions of flip-flops. Due to physical wiring delays, there will be minute differences in the times the clock arrives at different flip-flops—this is clock skew. Chip design engineers must carefully plan a "clock tree," using equal-length wires and repeater drivers, to ensure the clock signal reaches all flip-flops as close to simultaneously as possible.

This chapter is the physical starting point of "memory." 8 flip-flops form 1 byte (Chapter 15), a large array of flip-flops plus an address decoder forms RAM (Chapter 16), registers (made of flip-flops) are the fundamental storage units of the CPU (Chapter 17), and the memory managed by the operating system is, at its core, managing the allocation and reclamation of arrays of flip-flops (Chapter 22).

But all of this began with the two NOR gates in this chapter locked onto each other. Not the material, but the structure. Current circulating in a loop can remember. This is the physical essence of memory.

Chapter 15: Bytes and Hex

15.1 The Core Problem: Too Many Bits, How to Manage Them?

Chapter 14 built the flip-flop—it can store 1 bit. But 1 bit cannot store any meaningful amount of information. To store a single letter, a number, or an instruction, you need many bits.

This chapter does two things: groups 8 bits into a single unit, named the "byte"; and then uses hexadecimal to abbreviate this group of bits. These two conventions define the memory addressing method of all computers for the next eighty years.

15.2 Why 8 Bits?

8 bits can represent 256 different values. This number is enough to cover English letters (52 total for both upper and lower case), numbers (10), punctuation marks (about 30), and various control characters—this is precisely the code space of ASCII encoding. If a byte were 7 bits, 128 combinations would be too few. If a byte were 9 bits, 512 combinations would be too many, wasting code space. 8 bits is the optimal compromise.

Memory is a linear array composed of billions of bytes, and each byte has a unique numerical address. A 32-bit CPU can address up to 4GB of bytes at most, and a 64-bit CPU can theoretically address a range so vast it is nearly infinite—this is not your hard drive space; it is the physical memory limit.

15.3 Hexadecimal: The Human-Readable Shorthand for Binary

Writing 8 bits in binary is far too long—11111111 requires eight characters. Converting to decimal requires calculation and is not intuitive. Hexadecimal solves this perfectly: 4 bits correspond exactly to 1 hexadecimal digit, and 8 bits correspond exactly to 2 hexadecimal digits. A-F are not letters; they are numbers. A=10, B=11, C=12, D=13, E=14, F=15.

Any single 8-bit byte can be represented compactly by 2 hexadecimal digits. This is why memory addresses, debuggers, and machine code disassembly all use hexadecimal—it is the optimal compromise between binary and human readability. The computer internally always uses binary; hexadecimal is merely shorthand written on screens and printed on paper for humans to read.

15.4 Endianness: How Multi-Byte Data Is Arranged

When a number exceeds one byte, there are two conventions for the storage order of the multiple bytes into which it is split inside memory. Big-endian places the high-order byte at the low address; network protocols typically use this. Little-endian places the high-order byte at the high address; x86 and ARM (in modern mode) use this. When the same piece of multi-byte data is transmitted between different machines, completely different numbers will be read if the byte order is not converted.

15.5 The Physical Process of Reading Memory

To read the byte at a certain address, the CPU first outputs the address of that byte onto the address bus, then issues a "read" control signal. Based on the address, the memory controller activates the corresponding word lines and bit lines, outputting the values of the 8 flip-flops corresponding to that address onto the data bus. The time from when the address is output to when the data stabilizes and returns is called the "memory access latency."

The same string of hexadecimal digits, interpreted at different bit widths, will yield different meanings—the context determines the parsing method. The same byte can be a number, a character, or the operation code of a machine instruction in different contexts. This is why a disassembler needs to know the boundary between the code segment and the data segment.

15.6 Summary

This chapter is the bridge between data representation and the memory system. How to organize billions of bytes into addressable RAM (Chapter 16), how ASCII code places 7-bit encoding inside 1 byte (Chapter 20), and how multi-byte data types like floating-point numbers are arranged in memory (Chapter 23).

But all of this began with the two simple conventions of this chapter: 8 bits form 1 byte, and 4 bits are abbreviated into 1 hexadecimal character. Not physical laws, but conventions. Yet these two conventions have outlived most physical laws.

Chapter 16: An Assemblage of Memory

16.1 The Core Problem: Among 64K Flip-Flops, How to Precisely Find One?

Chapter 14 built the D flip-flop—it can store 1 bit. Chapter 15 grouped 8 flip-flops into 1 byte. Now, suppose you want to build a memory that can store 4096 bytes; you would need 32768 flip-flops. Here comes the problem: if the CPU wants to read the byte at address 1080, how does it pick out exactly those 8 flip-flops from among these thirty thousand?

This chapter solves the core engineering problem of memory: addressing. It introduces the address decoder—a circuit that activates the corresponding storage unit based on an address number. This chapter is the final building block for the "automation" of Chapter 17—before the CPU can be born, its memory must first be built.

16.2 The Address Decoder: "Lighting Up" the Corresponding Storage Unit Based on an Address Number

The input to an address decoder is address lines (N wires), and the output is select lines (2^N wires). For any combination of inputs, one, and only one, select line is activated. All the flip-flops connected to that activated select line simultaneously "open their doors"—allowing data to be read out or written in.

Take the simplest example: 10 address lines can decode into 1024 select lines. Each select line is connected to an 8-bit register (8 D flip-flops), and each register has a unique address. When the CPU outputs address 0, the 0th select line is activated, the 0th register opens its door, and data flows out onto the data bus. When the address is 1, the 1st register opens. And so on.

This is the most primitive form of computer memory—rows of numbered flip-flops, with the address decoder deciding which row is "selected."

16.3 The Three Buses of RAM and the Read/Write Process

The entire RAM connects to the outside world through three buses: the address bus tells the RAM which address to access, the data bus carries the actual data being read or written, and the control bus sends read/write commands.

Read process: The CPU places the address on the address bus and asserts the read signal (RD). The address decoder inside the RAM activates the corresponding storage unit, and that unit's data is placed onto the data bus. The CPU samples the data from the data bus. Read complete.

Write process: The CPU places the address on the address bus, places the data to be written onto the data bus, and asserts the write signal (WR). The address decoder inside the RAM activates the corresponding storage unit, and the value on the data bus is written into that unit. Write complete.

The tri-state gate is the key to bidirectional data bus transmission. The same wire cannot be driven by both the CPU and the RAM simultaneously—that would cause a short circuit, or even burn out the chip. The output of a tri-state gate can be in one of three states: logic 1 (driving a high voltage), logic 0 (driving a low voltage), or high-impedance (driving no voltage at all, effectively disconnected). During a read operation, the RAM output is enabled and the CPU input driver is turned off (high-impedance). During a write operation, the CPU drives the bus and the RAM input is enabled. The two sides never "speak" at the same time.

16.4 Chip Select: The "Roll Call" for Multiple RAM Chips

A computer typically has more than one RAM chip. When the CPU sends out an address, all RAM chips "hear" this address, but only the one chip that is "chip-selected" will truly respond. Chip select is essentially an extension of the address bus—the higher address lines, through a decoder, generate the chip select signal for each chip, while the lower address lines perform the addressing inside the chip. To the CPU, all the RAM chips form a single, continuous logical address space, and it doesn't need to care which chip physically holds the data.

16.5 SRAM and DRAM: The Fundamental Difference Between Two Storage Technologies

The previous sections assumed all memory was implemented using flip-flops. This type of storage unit, which retains data as long as power is supplied, is called Static RAM (SRAM). SRAM is fast but expensive—each cell requires 6 transistors, taking up a large area and consuming high power. The CPU's internal caches use SRAM.

The other type of memory uses a tiny capacitor to store charge—capacitor charged = 1, discharged = 0. This is called Dynamic RAM (DRAM). A DRAM cell only needs 1 transistor plus 1 capacitor, taking up a small area, consuming low power, and costing very little. But capacitors leak—the charge will leak away within a few milliseconds, so the data must be periodically refreshed (read out and written back) to be maintained. Main memory uses DRAM. When power is lost, all data is lost—this is why your computer's memory is cleared when you turn off the power.

The refresh operation of DRAM is handled automatically by the memory controller, but during a refresh cycle, the memory cannot respond to CPU read/write requests—this creates refresh latency. This is why memory latency cannot be as precisely predicted as CPU clock cycles.

16.6 The Memory Hierarchy: Trading Off Capacity and Speed

SRAM is fast but expensive; DRAM is cheap but slow. Modern computers combine both into a memory hierarchy: CPU Registers (fastest, smallest) → L1 Cache (SRAM, tens of KB) → L2 Cache (SRAM, hundreds of KB to a few MB) → L3 Cache (SRAM, a few MB to tens of MB) → Main Memory (DRAM, a few GB to hundreds of GB) → SSD / Hard Disk Drive (permanent storage, hundreds of GB to several TB). The closer to the CPU, the faster and more expensive; the further from the CPU, the slower and cheaper.

This chapter is the physical implementation of the storage system. The CPU interacts with memory through the address bus and data bus—fetch, decode, execute, memory access, write back (Chapter 17). Bus protocols and arbitration mechanisms (Chapter 21). The virtual memory, paging, and cache management managed by the operating system—all these mechanisms manage the arrays of flip-flops described in this chapter (Chapter 22).

But all of this began with the simplest question of this chapter: among thirty thousand flip-flops, how to precisely select those 8? The answer is a decoding tree—input an address, output one unique select line. This is the physical essence of computer memory addressing.

Chapter 17: Automation

All the previous sixteen chapters have assembled all the parts. The adder can calculate, the flip-flop can remember, and the RAM can store. But these parts are still just parts—they cannot move on their own. You need a person sitting in front of the computer, manually flipping switches, setting addresses, pressing a "write" button, and then waiting to see the result. This is not a "computer"; this is just "a pile of circuits that can do arithmetic."

To turn this pile of circuits into a true computer, one fundamental question must be answered: How do you tell the computer "what to do," and then have it execute automatically, without a person manually intervening at every single step?

17.1 The Von Neumann Architecture: Instructions and Data Mixed in the Same Memory

The answer comes from John von Neumann. He proposed an architecture still in use today: placing instructions and data in the same memory and accessing them through the same bus. Before this, computing was "programmed" by plugging and unplugging wires or flipping switches—changing one operation required re-wiring. The von Neumann solution was: use some bytes in memory to represent "what operation to do," and other bytes to represent "whom to operate on."

The format of machine code is simple: an opcode plus operands. The opcode is a number that tells the CPU what operation to execute. The operand is the target of this operation—it can be a memory address, an immediate value, a register number, etc. The CPU fetches this byte from memory, checks the opcode, and then executes the corresponding operation.

Instructions and data sharing the same bus is the greatest weakness of the von Neumann architecture. In the same clock cycle, the CPU can either fetch an instruction from memory, or read/write data from/to memory—it cannot do both simultaneously. This is the "von Neumann bottleneck."

17.2 The Instruction Cycle: Fetch, Decode, Execute

The rhythm of the CPU's automatic operation is a continuously repeating cycle. Fetch: The CPU outputs the current value of the Program Counter onto the address bus and reads one byte from that address in memory—this is the opcode of the next instruction. The Program Counter automatically increments by 1 to point to the next address. Decode: Based on the opcode, the Control Unit determines what operation this instruction is and sets the corresponding control signals. Execute: Based on the decoding result, the Control Unit commands the ALU to perform a calculation, or commands a register to load data, or commands the memory to perform a read/write operation. If it is a jump instruction, the value of the Program Counter is modified. After execution is complete, it automatically enters the next fetch cycle.

This is like a never-ending robot: fetch one instruction, execute it, fetch another, execute it—until the power is cut or a halt instruction is executed.

The Program Counter is the CPU's "signpost"; it always holds the address of the next instruction to be executed. When executing sequential instructions, it automatically increments by 1 each cycle. When encountering a jump instruction, it is forcibly modified to the jump target address. It is essentially a register with an auto-increment function—this is the autopilot that allows the CPU to execute instructions automatically.

17.3 The Control Unit: Hardwired Control vs. Microcoded Control

The Control Unit is the "brain" of the CPU. Early CPUs used hardwired control, directly implementing the instruction decoding logic with a large number of logic gates—fast execution speed, but extremely difficult to modify and extend. Later CPUs increasingly adopted microcoded control, decomposing complex instructions into a sequence of internal micro-operations (stored in ROM). The control logic is relatively simpler, modification and extension are more flexible, but the speed is slightly slower. Modern CPUs typically use a hybrid approach: frequently used instructions are implemented in hardwired logic to guarantee speed, while complex instructions are implemented in microcode to guarantee flexibility.

When a single machine instruction is executed, it internally requires multiple micro-operation steps—the Program Counter increments by 1, the address bus outputs the address, the memory sends out data, the ALU performs addition, the result is written back to a register. These micro-operations need to be completed over multiple clock cycles. Modern RISC processors typically pipeline instruction execution, dividing it into five pipeline stages: "Fetch, Decode, Execute, Memory Access, Write Back." Ideally, one instruction is completed per clock cycle.

17.4 The Instruction Set: The RISC vs. CISC Divide

The instruction set is the collection of all instructions a CPU supports. Some CPUs adopt a CISC design: many instruction types, single instructions with complex functionality, implemented with microcode. Other CPUs adopt a RISC design: few instruction types with fixed formats, implemented with hardwired logic—a single instruction only performs a simple operation, and complex tasks are completed by combining simple instructions. This divide has lasted for decades and continues to evolve to this day.

This chapter is the final assembly of the entire book's hardware portion. Relays → Vacuum Tubes → Transistors → Integrated Circuits; the physical implementation continues to evolve, but the logical architecture remains unchanged. The comparison of the 8080 and the 6800, to understand the origins of modern CPU instruction sets. The operating system manages the resource allocation and scheduling on top of the von Neumann architecture. The compiler translates your code into a sequence of machine instructions for the von Neumann architecture.

But all of this began with the simple loop of this chapter: fetch, decode, execute. Fetch, decode, execute. The reason a computer can be "automatic" is not because a clever brain is commanding it, but because this loop never stops. Until you turn off the power.

Chapter 18: From Abacus to Chips

The previous seventeen chapters progressed from a flashlight to the von Neumann architecture. But the component we have been using all along has been the relay—that mechanical device that uses an electromagnet to attract an armature. Real-world computers have not been built from relays for a very long time. From relays to modern chips, what happened in between?

This chapter is a "historical fast-forward." It condenses the complete evolutionary history from mechanical calculators to microprocessors into a single chapter, allowing the reader to see clearly: all technological progress is, at its core, doing the same thing—building a switch that is faster, smaller, and more energy-efficient. The logic hasn't changed, the architecture hasn't changed; what has changed is the physical implementation.

18.1 Mechanical Switches → Relays: From Finger to Electromagnet

A relay is still a mechanical switch. The armature needs to physically move, and inertia limits its speed—at most, it can switch dozens of times per second. Contacts wear out, and arcs cause erosion. Relay-based computers required large maintenance teams—replacing contacts, adjusting armatures, cleaning dust. But they achieved, for the first time, the automation of "using electric current to control electric current." The principles of logic gates and flip-flops were all verified on relays.

18.2 Relays → Vacuum Tubes: From Mechanical Motion to Electron Flow

A vacuum tube has no mechanical motion—it relies purely on electrons flying through a vacuum. A tiny voltage change on the grid can control a large current flowing from the cathode to the anode. Switching speed jumped to tens of thousands of times per second, hundreds of times faster than a relay. But vacuum tubes burn out like light bulbs. ENIAC had 18,000 vacuum tubes, and on average, several burned out every day. Maintenance personnel pushed carts around inside the machine, replacing tubes back and forth. Vacuum tubes consumed enormous power, generated tremendous heat, and had a very short lifespan—but they proved that a "computer with absolutely no mechanical motion" was feasible.

18.3 Vacuum Tubes → Transistors: From Vacuum to Solid

The transistor replaces the vacuum with semiconductor material. It doesn't rely on electrons flying through a vacuum, but on electrons moving within a solid material. No filament, no need for heating, no bulbs burning out. The three electrodes of a transistor—emitter, base, collector—correspond exactly to the three electrodes of a vacuum tube—cathode, grid, anode. A tiny current on the base controls a large current flowing from the emitter to the collector.

Switching speed jumped to millions of times per second, power consumption dropped to one-thousandth of a vacuum tube, and size shrank to fit on a fingertip. Starting with the transistor, computers truly entered the "electronic solid-state" era—no moving parts, no vacuum glass tubes, only the flow of electrons within solid materials.

18.4 Transistors → Integrated Circuits: From Discrete Components to Cities on Silicon

An integrated circuit etches multiple transistors, resistors, and wires simultaneously onto a single silicon chip, no longer requiring the soldering of discrete components one by one. A circuit that originally required an entire circuit board to hold can now be compressed into a silicon chip the size of a fingernail.

Small-scale integration has dozens of transistors per chip, medium-scale has hundreds, large-scale has thousands, and very-large-scale has over a million. When millions of transistors are integrated onto a single silicon chip, it becomes possible to etch the entire von Neumann architecture onto a single piece of silicon—this is the microprocessor.

18.5 Integrated Circuits → Microprocessors: The Entire CPU on a Single Chip

A microprocessor is simply the entire CPU integrated onto a single silicon chip. The Intel 4004 was the first commercial microprocessor, 4-bit, with 2,300 transistors. Starting with the 4004, Moore's Law began to drive progress—the number of transistors on an integrated circuit doubles approximately every two years. This empirical observation held true for half a century.

From gears to silicon wafers, the essence of a switch has always been "using a small signal to control a large signal." What has changed is only the physical carrier: mechanical lever → electromagnet → vacuum electric field → semiconductor electric field. Each generation is faster, smaller, more energy-efficient, and more reliable.

Transistor sizes are approaching their physical limits—quantum tunneling effects at the atomic level mean that sizes can no longer be shrunk indefinitely. Every transistor generates heat during its switching process, and with billions of transistors packed into an area the size of a fingernail, the heat density has surpassed that of an electric stove. Heat dissipation is one of the core bottlenecks in modern chip design—it's not that the switch cannot be faster, but that the heat dissipation cannot keep up.

This chapter is the endpoint of the hardware evolution history, and also the starting point for the chapters that follow. The specific implementations and instruction set comparisons of two typical microprocessors (Chapter 19), the standard channels for a microprocessor to communicate with the outside world (Chapter 21), and the operating system that manages the vast von Neumann machine constructed from silicon switches (Chapter 22).

But all of this began with that single "click" of a relay armature. Coil energized, armature attracted, contact closed—this is the physical starting point of the computer. From electromagnet to semiconductor electric field, from mechanical motion to electron flow, from discrete components to a billion transistors on a silicon chip. The logic hasn't changed. The switch has.

Chapter 19: Two Classic Microprocessors

Chapter 17 presented the abstract model of the von Neumann architecture—the Control Unit, ALU, registers, buses. But that was just a block diagram. Inside a real CPU, how are the registers organized? What does the instruction set look like? How do you write a program using these instructions?

This chapter selects two iconic 8-bit microprocessors from the 1970s for dissection: the Intel 8080 and the Motorola 6800. They are the direct ancestors of modern CPUs. The x86 architecture evolved from the 8080, and the ARM architecture is ideologically closer to the 6800. Understanding these two chips means you understand half of the genetic code within a modern CPU.

19.1 The Intel 8080 — The Progenitor of x86

The 8080 has seven 8-bit registers. The Accumulator, A, is the primary arithmetic register; the results of all arithmetic and logic operations are placed here. The general-purpose registers B, C, D, E, H, L can be used individually. Among them, H and L can be combined into the HL register pair, used as a 16-bit memory addressing pointer. There are two additional dedicated 16-bit registers: the Program Counter, PC, which points to the address of the next instruction, and the Stack Pointer, SP, which points to the top of the stack.

The 8080's instruction set includes data transfer instructions (MOV), arithmetic operation instructions (ADD, SUB), logic operation instructions (ANA, ORA), jump and call instructions (JMP, CALL, RET), and input/output instructions (IN, OUT). The addressing modes include immediate addressing (the operand is written directly within the instruction), register addressing (the operand is in a register), and register indirect addressing (the memory address pointed to by HL).

Every machine instruction is composed of one opcode byte and zero to two operand bytes. The format of the 8080's MOV instruction: the upper 5 bits of the opcode encode the MOV operation, and the lower 3 bits encode the source register and destination register numbers. All MOV instructions fall within the range of 0x40-0x7F. This "bit-field encoding" allows the CPU to check only the upper few bits during decoding, which is much faster than a lookup table by the full byte—this is a routine design technique in modern instruction sets.

19.2 The Motorola 6800 — The Pioneer of Clean Design

The 6800 has only three 16-bit registers: the Program Counter, PC; the Stack Pointer, SP; and the Index Register, IX; along with two 8-bit accumulators, A and B. It has far fewer registers than the 8080, but its addressing modes are richer—the 6800 supports immediate addressing, direct addressing (a 16-bit absolute address written directly in the instruction), extended addressing, indexed addressing, and relative addressing.

Another important characteristic of the 6800 is that all I/O devices are mapped into the memory address space—a program can access and modify I/O registers using memory read/write instructions. This "memory-mapped I/O" concept was later widely adopted.

19.3 A Key Comparison of the Two Chips

Dimension	Intel 8080	Motorola 6800
General-Purpose Registers	7 x 8-bit	2 x 8-bit
Dedicated Registers	SP, PC	SP, PC, IX
Number of Instructions	~72	~78
Addressing Modes	4	6
I/O Method	Isolated I/O ports	Memory-mapped I/O
Architectural Descendants	x86 family	68000 series → RISC thinking influenced ARM

19.4 The Stack: The Unsung Hero of Subroutine Calls

The stack is a Last-In, First-Out (LIFO) data structure, implemented using the Stack Pointer, SP. When calling a subroutine, the hardware automatically pushes the return address onto the stack. When executing RET, it automatically pops the return address from the stack and restores it into the Program Counter. During an interrupt, the context is similarly automatically pushed onto the stack to save the state. The stack makes subroutine nesting and recursion possible—every CALL pushes the current position onto the stack, every RET pops back to the previous level. If the calls go too deep and cause a stack overflow, it will corrupt data outside the stack, leading to a program crash.

A seemingly simple instruction like MOV A,B internally requires multiple micro-operation steps: PC outputs to address bus → memory reads opcode → instruction register → decode → PC increments by 1 → content of B sent to A. A complex instruction like CALL requires even more steps. Modern CPUs pipeline these micro-operation steps to increase throughput.

19.5 The Cost of Backward Compatibility — The Genetic Code of x86

The 8080 gave rise to the 8086 (16-bit), the 8086 gave rise to the 80286 and 80386 (32-bit), all the way to today's x86-64 (64-bit). Each generation must be compatible with the instruction set of the previous generation. This means the latest CPU in 2026 can still internally execute most of the instructions of the 1974 8080. Backward compatibility built an ecological moat for Intel, but also made the x86 instruction set exceptionally complex—a modern x86 decoder needs to translate complex legacy instructions into internal RISC-style micro-operations for execution.

This chapter is a concrete instance of the internal structure of a CPU. The encoding standard when a CPU processes characters (Chapter 20), how a CPU accesses memory and I/O devices through buses (Chapter 21), the operating system that manages the CPU's process scheduling and memory allocation (Chapter 22), and the compiler that translates high-level languages into sequences of 8080/6800-style machine instructions (Chapter 24).

But all of this began with the register lists and instruction sets of the two chips in this chapter. Seven registers, seventy-odd instructions. This is the genetic code of the modern CPU.

Chapter 20: ASCII and a Cast of Characters

For the first nineteen chapters, the computer could only process numbers—binary numbers, hexadecimal addresses, machine instruction opcodes. But a computer cannot only deal with numbers. It needs to process text—your code, your documents, the characters you are reading right now. Turning letters into numbers is the core problem this chapter aims to solve.

20.1 7 Bits Encode 128 Characters

ASCII uses 7 bits to represent 128 characters, grouped by function. 0 to 31 are control characters—Carriage Return, Line Feed, Tab, Bell—they do not print any glyph, but control the behavior of the terminal. These characters are remnants from the teletypewriter era: Carriage Return sends

the print head back to the beginning of the line, Line Feed scrolls the paper. Even though your screen today has no physical print head, these two characters still control line breaks in all operating systems. 32 to 47 are punctuation marks and the space character. 48 to 57 are the digits 0 through 9, arranged consecutively. 65 to 90 are the uppercase letters A through Z, arranged consecutively. 97 to 122 are the lowercase letters a through z, arranged consecutively.

You only need to remember a few anchor points. Space is 32. '0' is 48, and digit characters are arranged consecutively from here; the ASCII code for 5 is 48 plus 5, which is 53. 'A' is 65, and uppercase letters are arranged consecutively from here; D is 65 plus 3, which is 68. 'a' is 97, and lowercase letters are arranged consecutively from here; d is 97 plus 3, which is 100.

The code point difference between uppercase and lowercase letters is 32. 'A'=65, 'a'=97, and 97 minus 65 equals 32. This is no coincidence—the ASCII code of an uppercase letter, when bit 5 (decimal value 32) is set to 1, becomes the corresponding lowercase letter. This design allows case conversion to be done with a single bitwise operation.

20.2 Text Is Numbers

A line of text on the screen is just a sequence of ASCII codes inside the computer. "Hello" equals 72, 101, 108, 108, 111. When each byte is output to the display memory or terminal, the character generator looks up the corresponding glyph based on its ASCII code and draws the corresponding pixels on the screen.

Text is numbers—numbers are flip-flop states—flip-flops are logic gates—logic gates are relays. This chain of mapping stretches from the two kids signaling with flashlights in Chapter 1 all the way to a single letter H on a screen.

Windows uses CR followed by LF to mark a line break; Unix, Linux, and macOS use LF. If you use Windows Notepad to open a Linux text file, all the content will be squished onto a single line—because Windows is waiting for an LF that will never come.

20.3 From ASCII to Unicode

ASCII is a 7-bit encoding, with a maximum of 128 characters. This is sufficient for English, but the accented letters of European languages and the special characters of Northern Europe already exceed its bounds. Thus, Extended ASCII emerged—using all 8 bits, adding an additional 128 characters, with various manufacturers defining them independently, making them mutually incompatible. This is completely insufficient for Chinese—there are tens of thousands of characters, and 8 bits will never be enough.

The solution is Unicode—an encoding system that "assigns a unique number to every single character in the world." Unicode allocates a unique code point for each character. UTF-8 is a variable-length encoding: ASCII characters remain unchanged, still encoded in 1 byte; common European and Arabic characters use 2 bytes; East Asian Han characters use 3 bytes; rarely used ancient scripts and emoji use 4 bytes. UTF-8 is fully backward compatible with ASCII—English text is encoded identically under both ASCII and UTF-8.

ASCII is a single-byte character set—one character is exactly one byte. UTF-8 is a multi-byte encoding—one character can be 1 to 4 bytes. Many programs confuse "character length" with "byte length," and bugs will appear when processing non-English text: truncating a multi-byte character causes garbled text, and reversing a byte sequence destroys the UTF-8 structure. This is the most common pitfall when using Unicode.

The ASCII order places uppercase letters before lowercase letters. When sorting by ASCII, Z will appear before a, which differs from dictionary order. When comparing strings, you must distinguish between "binary comparison" and "dictionary order"—the latter requires case folding and accent handling based on the current language locale.

This chapter is the foundation of text representation. Character data is transmitted between the CPU, memory, and peripherals via the data bus (Chapter 21). The essence of a file system reading and writing files is reading and writing a sequence of ASCII and UTF-8 bytes (Chapter 22). A compiler parses source code strings into tokens and then compiles them into machine instructions (Chapter 24).

But all of this began with the simple convention of this chapter: 'A'=65, 'a'=97, space=32. These numbers are not physical laws, but conventions. Yet this convention allows computers all over the world to read the same piece of text. Characters are numbers. Numbers are flip-flops. Flip-flops are logic gates. Logic gates are relays. From a flashlight to the letter H, the circle is closed.

Chapter 21: The Bus

Previous chapters have mentioned the address bus, the data bus, and the control bus here and there, but they have always been treated as "connection lines that already exist." This chapter answers a question that has been deferred for a very long time: What exactly are these buses? When multiple devices are hung on the same bus, how do you prevent them from interfering with each other?

A bus is the "public address system" inside a computer. The CPU shouts an address on the address bus, all devices hear it, but only the device corresponding to that address will respond.

21.1 The Three Buses: Address, Data, Control

The address bus is unidirectional—signals only travel outward from the CPU. Through it, the CPU tells all devices, "The address I want to access right now is here." The number of address lines determines the addressing capability: 16 lines can address 64KB, 32 lines can address 4GB.

The data bus is bidirectional—data can flow from the CPU to memory or peripherals (a write operation), or flow in the reverse direction (a read operation). The number of data lines determines how many bits can be transmitted at once: 8 lines transmit 1 byte at a time, 64 lines transmit 8 bytes at a time. Bidirectional transmission is achieved using tri-state gates—at any given moment, only one device drives the bus; all other devices maintain a high-impedance state, effectively disconnected.

The control bus carries various command signals: Memory Read, Memory Write, I/O Read, I/O Write, Interrupt Request, Interrupt Acknowledge, Reset, and Clock.

21.2 Chip Select: The Bus's "Roll Call" Mechanism

Multiple devices are hung on the same address bus. When the CPU sends out an address, the signal reaches all devices simultaneously. But only the one device that is "chip-selected" will truly respond. The higher address lines, through a decoder, generate the chip select signal for each chip—when a chip's chip select signal is asserted, that chip "wakes up," receives the input from the lower address lines, and executes the read or write operation. The chip select signals for the other chips are inactive; they completely ignore the data changes on the bus.

Chip select is essentially an extension of the address bus—the higher address lines are responsible for "selecting the chip," and the lower address lines are responsible for "addressing within the chip." To the CPU, all devices form a single, continuous address space, and the fact that the data is physically scattered across different chips is completely transparent.

21.3 Memory-Mapped I/O: Accessing Peripherals as Memory

There are two ways for the CPU to communicate with I/O devices. Isolated I/O uses dedicated IN/OUT instructions, and the I/O address space is separate from the memory address space—the 8080 adopted this method. Memory-Mapped I/O maps the registers of peripherals to a certain region of the memory address space—reading and writing these special addresses is equivalent to reading and writing the peripheral registers. No dedicated I/O instructions are needed; ordinary memory access instructions can be used to control the peripherals.

Most modern mainstream processors adopt Memory-Mapped I/O—the graphics card framebuffer, network card registers, timers, and interrupt controllers are all mapped as "memory addresses," uniformly addressed and uniformly accessed.

21.4 DMA: Direct Memory Access, Bypassing the CPU

Under CPU control, reading a single byte from a disk requires first reading it into a CPU register, then writing it back to memory—the CPU is involved throughout the entire process, and each byte consumes CPU time. This is extremely inefficient when transferring large blocks of data. DMA (Direct Memory Access) allows a peripheral to bypass the CPU and exchange data directly with memory. The CPU first sets up the source address, destination address, and transfer length, then hands over full control to the DMA controller. During the DMA transfer, the CPU can continue executing other instructions. After the transfer is complete, the DMA controller sends an interrupt to notify the CPU. This is the foundation underlying modern high-speed I/O—NVMe SSDs and 10-Gigabit network cards.

21.5 Bus Arbitration and Serialization

DMA controllers, multi-core CPUs, and multiple bus master devices may simultaneously request the right to use the bus. The bus arbiter is responsible for deciding who gets to "speak" first. Modern PCIe buses use point-to-point serial connections paired with a switch for routing, no longer a shared parallel bus with devices hung all over it—this is the fundamental solution to the problem of multiple devices contending.

Early buses increased bandwidth by widening the data lines—8-bit to 16-bit to 32-bit to 64-bit. But more wires meant more complex routing and more severe clock skew. Modern high-speed interfaces have all switched to high-speed serial differential pairs—transmitting only 1 bit at a time, but at an extremely high frequency. Paired with differential signaling for noise immunity and dedicated encoding to eliminate clock skew, the actual effective bandwidth far exceeds that of the old parallel buses.

This chapter is a complete description of the internal communication of a computer. The operating system that manages the devices on the bus and provides a unified driver interface (Chapter 22). The floating-point unit that interacts with the CPU and memory through the bus (Chapter 23). The code generated by the compiler that accesses memory and I/O devices through the bus (Chapter 24).

But all of this began with the three sets of signal lines in this chapter: the address bus shouts "who," the data bus delivers "what," and the control bus decides "what to do." They are the nervous system inside the computer.

Chapter 22: The Operating System

The previous twenty-one chapters built everything from a flashlight all the way up to the von Neumann architecture, the microprocessor, and the bus. All the hardware is ready. But this pile of hardware has a fundamental problem: it cannot manage itself. Who uses which portion of memory? Which process gets the CPU first? Where are the files placed on the disk? The hardware itself does not solve these problems.

The essence of an operating system is a resource scheduler. It abstracts physical resources into logical resources, so that a programmer does not need to know the physical address of memory, the cylinder and track of a disk, or the interrupt number of a network card. They only need to care about "how does my program read and write files, and create processes." The four core functions of an operating system are process management, memory management, the file system, and device drivers.

22.1 Booting: From Pressing the Power Button to an OS Being Ready

After the power button is pressed, the CPU executes the first instruction hardwired into the hardware, jumping to the BIOS or UEFI firmware. The firmware performs a Power-On Self-Test, checking whether basic hardware like memory, keyboard, and disk are functional. It then scans the list of boot devices, finds the first bootable disk, and reads its Master Boot Record. The boot loader locates the operating system kernel file, loads it into memory, and jumps to the kernel's entry address. After the kernel takes over, it initializes the interrupt vector table, memory manager, process scheduler, file system, and device drivers. Once everything is ready, the kernel creates the first user-mode process and waits for user input. Your terminal window is the last program created during the operating system boot process.

22.2 System Calls: The Interface Between User Programs and the Kernel

The operating system kernel runs in privileged mode; user programs run in unprivileged mode—a user program cannot directly operate hardware, cannot access the memory of other processes, and cannot modify critical system data. If a user program needs to access a file, allocate memory, or send network data, it must request the kernel to do it on its behalf through the system call interface provided by the operating system.

When executing a system call, the application places the call number and parameters into designated registers, and executes a special instruction to trigger a software interrupt. The CPU immediately switches to kernel mode, and the kernel dispatches the corresponding handler function based on the call number. After processing, the kernel returns the result to the user program, and the CPU switches back to unprivileged mode. File reading and writing, memory allocation, process creation, network communication—all operations involving hardware and system resources ultimately go through system calls.

22.3 Process Management: Time-Sharing the CPU

A process is an instance of a running program. Each process believes it has the CPU to itself. The operating system achieves time-sharing through timer interrupts—Process A runs for a few milliseconds, the timer interrupt fires, the scheduler saves the register context of Process A, selects Process B from the ready queue, restores the context of Process B, and Process B continues running. The frequency of process switching is typically tens to hundreds of times per second, and the switching is so fast that humans perceive all programs as running "simultaneously."

Processes need means of communication and synchronization—pipes, message queues, shared memory, semaphores. When concurrently accessing shared data, mutual exclusion must be ensured through mechanisms like locks, otherwise the data will be corrupted under race conditions.

With every process switch, the operating system must save all the register values of the current process, switch the page table, and load the context of the new process. The time consumed by these operations is completely devoid of useful computation—frequent switching can cause a significant drop in system throughput.

22.4 Memory Management: Virtual Memory

Physical memory is limited; all processes cannot simultaneously occupy all of memory. Virtual memory decouples each process's address space from physical addresses. Each process sees its own private virtual address space; the program believes it has all of memory to itself.

The MMU inside the CPU automatically translates virtual addresses into physical addresses. Address mapping is organized in units of pages, and the MMU performs the translation based on the page table. An unmapped virtual page triggers a page fault exception; the operating system either swaps a page in from the disk, or denies access and terminates the process. When memory is insufficient, the operating system swaps infrequently used pages out to the swap area on disk.

The significance of virtual memory lies in isolation—Process A cannot access the memory of Process B. A wild pointer write will only crash its own process, not corrupt other processes or the kernel. As the system runs, a large number of small, non-contiguous free areas will appear in physical memory. Even if the total free space is sufficient, it may be impossible to allocate a single contiguous block of physical pages. The operating system mitigates this fragmentation problem through compaction and reorganization, or through discrete paging allocation.

22.5 The File System: Data Structures on Disk

A disk is a sequence of sectors. The file system is responsible for organizing these raw sectors into a hierarchical directory tree and named files. The file system defines an inode to store the metadata of a file—size, permissions, timestamps, data block pointers. A directory is a special mapping table from file names to inode numbers. When reading or writing a file, the operating system looks up the directory entries level by level based on the path name to obtain the inode number, and then locates the physical location of the data on the disk based on the data block pointers in the inode.

A file descriptor is a handle to an open file. Each process maintains its own file descriptor table. A process reads and writes files through file descriptors, without needing to care about the specific locations of inodes and data blocks—this is handled by the kernel's file system driver.

22.6 Device Drivers: Everything Is a File

Every hardware device has its own communication protocol and control register layout. A device driver is a kernel module that provides a unified read/write interface to upper-layer applications, while translating standardized requests into specific sequences of I/O operations to control the particular hardware.

Device drivers standardize the operations of peripherals into file operation interfaces like open, read, write, close—this is the "everything is a file" design philosophy. A user program can operate a disk by reading and writing /dev/sda, operate a terminal by reading and writing /dev/tty, and discard data by reading and writing /dev/null. This unified file interface is the most profound design legacy of the Unix operating system.

This chapter is the bridge between computer hardware and application programs. The operating system's saving and restoring of floating-point context (Chapter 23). The object code generated by the compiler invokes the system call interface (Chapter 24).

But all of this began with the simple abstraction of this chapter: turning physical resources into logical resources. The CPU becomes processes, memory becomes a virtual address space, the disk becomes a file tree, and peripherals become files. The operating system is not hardware, nor is it an application program; it is a thin layer of software sandwiched between the two. This layer of software allows programmers to no longer need to know everything about the hardware, but only need to know how to call the system interface. This is the entire significance of the operating system.

Chapter 23: Fixed Point, Floating Point

For the previous twenty-two chapters, the computer only processed integers—represented using two's complement. But in reality, there are vast amounts of non-integer calculations: a temperature of 36.5 degrees, the value of Pi 3.14159, the speed of light 299,792,458 meters per second. Integers cannot meet these needs.

This chapter answers how a computer represents and processes fractional numbers. The core solution is floating-point numbers—the binary version of scientific notation. The answer to all questions about why 0.1 plus 0.2 does not equal 0.3 lies within the IEEE 754 standard.

23.1 Fixed-Point Numbers: The Decimal Point Is Fixed

A fixed-point number pre-agrees on the position of the decimal point. All numerical values are scaled by a fixed multiplier—the displayed value of the data equals the stored integer value divided by this multiplier. Addition and subtraction are exactly the same as with integers, requiring no extra logic. Multiplication and division, however, require compensating for the scaling factor.

Fixed-point numbers are widely used in fields like digital signal processing—embedded processors without a Floating-Point Unit often use fixed-point arithmetic to simulate fractional numbers. But the fatal weakness of fixed-point numbers is their fixed dynamic range—numbers whose absolute value is smaller than the minimum precision cannot be represented, and numbers whose absolute value exceeds the upper limit will overflow immediately.

23.2 Floating-Point Numbers: Scientific Notation in Binary

Floating-point numbers allow the decimal point to "float," supporting an enormous range of both very large and very small values by separating the significant digits from the exponent. This is the same concept as decimal scientific notation—expressing a value as a mantissa multiplied by the base raised to the power of an exponent. Floating-point numbers encode this concept into a fixed bit width using standardized fields: 1 bit for sign, several bits for the exponent, and several bits for the mantissa.

23.3 IEEE 754 Single-Precision Floating-Point Numbers

A single-precision floating-point number is 32 bits total, divided into three fields. The sign bit, S, occupies 1 bit—0 for positive, 1 for negative. The exponent, E, occupies 8 bits. The mantissa, F, occupies 23 bits, storing the significant digits after the decimal point. The decoding rules depend on the values of E and F.

E=0 and F=0 represents positive zero or negative zero. E=0 and F not zero is a denormalized number—used to represent extremely small values. E between 1 and 254 is a normalized number—this is the most common, normal floating-point number; its value equals negative 1 to the power of S, times 2 to the power of (E minus 127), times 1.F. E=255 and F=0 represents positive infinity or negative infinity. E=255 and F not zero is NaN—Not a Number, the result of operations like zero divided by zero.

In a normalized number, an implicit 1 precedes the mantissa. This 1 does not occupy a storage bit, magically granting one extra bit of precision. In a denormalized number, an implicit 0 precedes the mantissa, allowing the representation of values even smaller than the smallest normalized number—this is the mechanism of "gradual underflow," preventing results that are too small from immediately rounding to zero.

23.4 The Precision Problem: Rounding Error

The decimal fraction 0.1 is an infinitely repeating fraction in binary. When represented using a 32-bit floating-point number, it can only be truncated to 23 mantissa bits, so 0.1 can only be an approximation inside the computer. The result of 0.1 plus 0.2 differs from 0.3 by roughly 10 to

the power of negative 17—precisely because of this truncation error.

The 23-bit mantissa of single-precision corresponds to roughly 7 decimal significant digits; the 52-bit mantissa of double-precision corresponds to roughly 16 decimal significant digits. Exceeding the range of significant digits requires rounding. IEEE 754 defaults to the "round to nearest, ties to even" strategy to cancel out positive and negative bias across a large number of calculations.

Floating-point arithmetic does not satisfy the associative law or the distributive law—a plus b plus c may not equal a plus b plus c if the grouping differs, and a times the quantity b plus c may not equal a times b plus a times c. Every operation produces a rounding error, and the accumulation path of the error depends on the order of operations. A compiler's optimization options may reorder floating-point operations and thereby change the calculation result. In high-precision scientific computing, special attention must be paid to numerical stability—avoiding large numbers swallowing small numbers, and avoiding catastrophic cancellation when two close numbers are subtracted.

Double precision is 64 bits—1 sign bit plus 11 exponent bits plus 52 mantissa bits—with an exponent bias of 1023, yielding roughly 16 significant decimal digits. Quadruple precision, 128 bits, provides roughly 34 significant digits. The x86 FPU internally uses 80-bit extended precision to prevent precision loss in intermediate results.

23.5 Floating-Point Overflow and Exceptions

The possible exceptional results of floating-point operations include: a result exceeding the maximum representable value returns infinity; a result smaller than the minimum representable value returns zero or a denormalized number; division by zero returns infinity or NaN; an invalid operation returns NaN. NaN has a special contagious property—the result of any operation with NaN is NaN, which allows errors to be traced throughout a chain of numerical computation.

This chapter is a complete description of numerical computation. The compiler translates floating-point operations in high-level languages into IEEE 754 machine instructions (Chapter 24). The parallel floating-point computing capability of the GPU is the foundation of modern graphics and AI computation (Chapter 25).

But all of this began with the simple convention of this chapter: 32 bits split into three segments—1 bit for sign, 8 bits for exponent, 23 bits for mantissa. This is the complete reason why 0.1 plus 0.2 does not equal 0.3. It is not a bug in the computer, but a mathematical property of binary itself. A finite number of bits can never contain an infinitely repeating fraction. This is the floating-point number.

Chapter 24: High and Low-Level Languages

The first twenty-three chapters all dealt with machine code—those binary sequences composed of opcodes and operands that the CPU can execute directly. To program in machine code, you need to memorize the numerical encoding of every instruction, manually calculate the target address of every jump, and manually allocate the memory location for every variable. If the target address of a branch is calculated incorrectly, the entire section of the program must be rewritten.

How do you move from machine code to a human-readable programming language? How does a compiler translate human-readable code into machine code? This chapter is the theoretical starting point leading to modern programming languages.

24.1 Assembly Language: Giving Machine Code a Name

Assembly language does one thing that is extremely simple yet extremely important: it gives every machine instruction a human-readable name. The addition opcode is no longer a number, but ADD. The jump instruction is no longer a number, but JMP. Registers and memory addresses can also be given names, no longer requiring the manual calculation of address offsets.

Assembly language and machine code have a one-to-one correspondence. Each assembly instruction corresponds exactly to one machine instruction. It merely clothes machine code in a human-readable garment, without adding any layer of abstraction. The assembler is responsible for translating these symbols into the corresponding machine code.

24.2 High-Level Languages: From "How to Operate Hardware" to "What to Calculate"

Assembly is just another way of writing machine code. To truly increase programming efficiency, a more fundamental abstraction is needed: allowing the programmer to no longer care about register allocation, instruction selection, or address offsets, and only needing to express "what I want to calculate."

High-level languages provide variable names and data types—no longer needing to manually manage memory addresses. They provide arithmetic expressions—one line of expression replaces multiple assembly instructions. They provide control structures—if/else, for/while, replacing manual jump instructions. They provide functions and procedures—encapsulating and reusing code, replacing manual stack push and pop operations.

The programmer writes x=a+b, and the compiler automatically completes the following: deciding which register or memory location a and b should be placed in, generating LOAD instructions to fetch a and b into registers, generating an ADD instruction to perform the addition, generating a STORE instruction to place the result into the location of x. The programmer does not need to care about any of these details at all.

24.3 The Compiler: The Translation Process from High-Level Language to Machine Code

Compilation is the process of translating a high-level language source file into machine code, typically divided into several phases.

Lexical analysis chops the source code string into individual tokens—the keyword if, the identifier x, the operator plus sign, the literal 123. A token is the smallest language unit the compiler processes.

Syntax analysis organizes the sequence of tokens into a syntax tree based on the grammar rules of the language. This step verifies the syntactic correctness of the program and produces an Abstract Syntax Tree, where each node of the tree represents a syntactic structure of the language.

Semantic analysis checks the semantic rules within the syntax tree—whether a variable has been declared, whether types match, whether the arguments of a function call are valid. This step maintains a symbol table recording the declaration information of all variables and types.

Intermediate code generation and optimization converts the syntax tree into an internal Intermediate Representation of the compiler and performs various optimizations on this intermediate representation—constant folding, dead code elimination, loop optimization, inline expansion. The intermediate representation is independent of any specific programming language and any specific CPU architecture. The optimizer performs platform-independent optimizations on the intermediate representation, and then the backend translates the optimized intermediate representation into platform-specific machine instructions.

Target code generation translates the optimized intermediate representation into machine instructions for a specific CPU architecture, performing backend processing such as register allocation and instruction scheduling.

24.4 The Interpreter: Don't Compile, Execute Directly

An interpreter does not take the path of "compiling into machine code," but directly reads the source code, analyzes it, and then executes it on the spot. The cycle from analysis to execution is repeated for each line of code executed. Interpreted execution is slower than compiled execution—because the same code must be re-analyzed and re-translated each time it is encountered. But an interpreter is simpler to develop, and allows runtime code modification and interactive debugging.

Many modern languages adopt a hybrid approach: first compile to bytecode, then execute with a bytecode interpreter. Java's JVM and Python's CPython both adopt this model.

The more errors a compiler can check at compile time, the fewer runtime crashes there will be. The type system is the primary tool for catching compile-time errors. C's type checking is relatively lenient—many implicit conversions and pointer operations can pass compilation but are unsafe. Rust's type system is far stricter—ownership and lifetimes eliminate use-after-free and data races at compile time. Compile-time checking moves from "probabilistic protection" toward "mathematical proof."

Compilers provide different optimization levels: level zero, no optimization, suitable for debugging and fast compilation; level one, basic optimization; level two, standard optimization; level three, aggressive optimization—aggressive inlining and vectorization; size optimization, optimizing for size. The more aggressive the optimization, the slower the compilation, but the faster the execution. Use level zero for fast iteration during development, and use level two or three to pursue performance during release.

This chapter is the theoretical foundation of programming languages. GPUs perform large-scale parallel computation through high-level languages (Chapter 25). Compiler principles, operating systems, and computer architecture all rely on the conceptual foundation established in this chapter.

But all of this began with the simple invention of assembly language: giving machine code a name. ADD replaces a string of numbers; JMP replaces another string of numbers. From names to expressions, from expressions to functions, from functions to type systems—each step allows the programmer to move further from the machine, and closer to thought. This is the entire evolutionary history of programming languages.

Chapter 25: The Graphical Revolution

For the first twenty-four chapters, the computer's input and output relied mainly on the keyboard and a command-line terminal—type text in, text comes out. But the core interactive mode of a modern computer is the graphical user interface: windows, buttons, icons, a mouse pointer, photos, videos. What exactly are these images inside the computer? How do they go from 0s and 1s in memory to the dazzling, colorful pixels on a screen?

25.1 The Pixel: The Smallest Unit of an Image

A display screen is divided into a giant rectangular grid; each cell of this grid is called a pixel. The screen resolution is the size of this grid—1920×1080 means 1920 columns horizontally and 1080 rows vertically, totaling roughly 2.07 million pixels. Each pixel can independently display one color.

The simplest black-and-white display only needs 1 bit—0 is black, 1 is white. A grayscale display uses one byte to represent 256 levels of gray. A color display uses three bytes to respectively represent the intensity of the three primary colors: Red, Green, and Blue—this is RGB. Modern displays typically use 24-bit true color per pixel, 8 bits each for R, G, and B, totaling roughly 16.77 million colors.

25.2 The Framebuffer: The Screen's "Shadow" in Memory

The framebuffer is a contiguous block of memory that holds the color values of all the pixels on the screen. Every single pixel on the screen corresponds to one or more storage units in the framebuffer. To change the color of a specific pixel on the screen, you only need to modify the value at the corresponding address in the framebuffer. The simplest framebuffer layout is a linear arrangement—starting from the top-left corner of the screen, storing the color value of each pixel row by row, from left to right.

For a 1920×1080, 24-bit color screen, one frame requires roughly 6.2MB of video memory. At a 60Hz refresh rate, the graphics card must read roughly 373MB of data per second from the framebuffer and send it to the display. Modern games and videos require even larger amounts of data—because the content of each frame is different, and an entirely new frame must be regenerated every 16.7 milliseconds.

25.3 The Character Generator: From ASCII Code to Pixel Pattern

In the command-line era, the screen displayed text using a character generator—a hardcoded ROM storing the pixel pattern corresponding to each ASCII character. To display the letter 'A', the terminal fetches the glyph bitmap corresponding to 'A' from the character generator—an 8×8 or 8×16 pixel matrix—and then writes the color values of the corresponding pixels into the appropriate positions in the framebuffer. The character generator made displaying text incredibly efficient—you didn't need to transmit a separate image for each letter, only a single ASCII code.

25.4 The GPU: Taking Over Graphics Computation from the CPU

Early computers only had a command-line interface—black background with white text, type in commands via the keyboard, and the computer prints the result on the next line after execution. The graphical user interface added windows, icons, menus, and a mouse pointer on top of this, with all elements drawn directly through the framebuffer. Dragging a single window requires redrawing the entire screen, and rendering a single anti-aliased font character requires performing edge blending calculations for every pixel. All of this requires immense graphics processing power.

Taking 2.07 million pixels, multiplied by the RGB calculations per pixel, multiplied by 60 frames per second, and handing it all over to the CPU to process is unrealistic. The GPU is a processor specifically designed for parallel graphics computation. Unlike the CPU's structure of a few large

cores, a GPU possesses hundreds or even thousands of small cores; each core is simple, but their sheer number is vast. The millions of pixels on the screen all require the same processing—calculating color, sampling textures, blending lighting—and all the small cores of the GPU can simultaneously process different blocks of pixels in parallel, making real-time graphics rendering possible.

The capability of modern GPUs has long since surpassed graphics rendering—scientific computing, password cracking, and AI training all rely on the massive parallel floating-point computing power provided by GPUs.

25.5 Bandwidth and Synchronization: The Engineering Challenges Behind Every Frame

The graphics card must continuously convert the contents of the framebuffer into a video signal and send it to the display. The higher the resolution, the greater the color depth, and the higher the refresh rate, the larger the required bandwidth. Furthermore, when the GPU renders a frame, it needs to repeatedly read and write data like textures, vertices, and depth—rendering a single frame may require reading an amount of intermediate data far greater than the final image itself, which is a tremendous test for both video memory bandwidth and capacity.

The refresh frequency of a display is fixed, but the time it takes for the GPU to generate each frame is not. If the GPU updates the framebuffer while the display is halfway through a refresh, the top half of the screen will show the old frame and the bottom half will show the new frame, causing a visible tear line on the screen. Vertical synchronization technology forces the GPU to wait until the display's current refresh cycle is completely finished before updating the framebuffer, eliminating tearing but potentially introducing latency. Modern adaptive synchronization technology allows the display's refresh rate to dynamically follow the GPU's frame rate, eliminating the conflict between a fixed refresh interval and tearing.

This chapter is the final destination of the entire book *Code*. We started with two kids signaling each other with a flashlight across a window—on represents 1, off represents 0, and this is the smallest unit of information. Then, step by step, we built logic gates, adders, flip-flops, memory, the CPU, the operating system, and programming languages. Finally, we return to the screen—those millions of pixels, each one occupying a storage address in the framebuffer, and the value of each one is a combination of 0s and 1s.

The image on the screen is the ultimate form, evolved all the way from the most primitive flashlight signal. From a flashlight to a screen, passing through switches, relays, logic gates, adders, flip-flops, the von Neumann architecture, the microprocessor, the bus, the operating system, and the compiler. All of these, in the end, converge in the framebuffer—what data is, what an instruction is, what an image is, are all, ultimately, a number on the address bus, a string of bits on the data bus, and a single pixel on the screen.

After reading this book, you will know that a computer is not magic. It is a logical machine built up layer by layer, composed of countless parts that can be understood. Every layer can be taken apart, and every layer can be comprehended. And you are the one who will build yet another layer yourself.

The Very Last Page of the Book: Write to Yourself

After you have finished reading everything, write these two lines on the very last page of your field manual:

> I started with one flashlight and one switch, and journeyed through relays, logic gates, adders, flip-flops, the von Neumann architecture, the bus, the operating system, and finally arrived at the graphical interface.
>
> I spent ____ days walking a path that took humanity a century to complete.

The Computer Science Canon, Volume I: A Chapter-by-Chapter Exegesis of *Code* · The End

> The Night Keeper's Book · File No. 08 · Volume I
> Authored by The Silent One / Collated by Kate / Archived by the Administrator